LECTURE NOTES

Advanced Algorithm Design

Complexity, NP-Completeness
and Approximation Algorithms

Prof. G. Pucci

Lectures 01–09
Including exercise sessions

Contents

Lecture 01: Course introduction and computational problems **1**
Advanced Algorithm Design (AAD) . . . 1

1 Advanced Algorithm Design (AAD) **1**
1.1 Objective: . . . 1
1.2 CourseTopics . . . 1
1.3 Course objective: . . . 1
1.4 Written Exam . . . 2
1.5 Pre-requisites . . . 2

2 Complexity Classes and Theory of Intractability (NP-Completeness) **2**
2.1 Computational Problem . . . 2
2.2 Complexity of a problem . . . 3
2.3 Problem Categories . . . 3
2.4 HALTING PROBLEM . . . 3
2.5 Intractable Problems . . . 3

3 Abstract Decision Problems **3**
3.1 EXAMPLE: Membership of integer key in a set . . . 4
3.2 Restriction to decision problems: excessive? . . . 4

4 Optimization Problem **4**
4.1 Example : Shortest Unweighted Path Problem (SUP) . . . 4
4.2 Example : K Unweighted Path Problem (KUP) . . . 4

Lecture 02: Encodings, formal languages and verification **5**

5 Lesson 1 Recap **6**

6 Classes of decision problems **6**

7 Encodings **6**
7.1 Encoding Choice . . . 7
7.2 Binary Encoding . . . 7
7.3 Unary Encoding . . . 7
7.4 Ripercussions on complexity analysis . . . 7
7.5 Coincise Encodings . . . 8

8 Formal Language Representation of Concrete Decision Problems **9**
8.1 Formal languages and concrete decision problems . . . 9
8.2 Polynomial-time decision problems and the complexity class P . . . 10
8.3 Search Problem . . . 10

9 Polynomial time verification **11**

10 Polynomial-time verification of Hamiltonian cycles **11**
10.1 Solution . . . 11
10.2 Verification . . . 11
10.3 Class NP and the relationship between P and NP . . . 13

Lecture 03: Reductions and NP-completeness **13**

11 Lesson 2 Recap **13**

12 Theorem and proof: P is a subset of NP **13**
12.1 Reasonable guess and evidence that P is not NP: 14

13 Reducibility between problems **15**
13.1 Example: reducibility between first and second degree equations 15

14 Order between Problems **15**
14.1 Polynomial time reducibility 15
14.2 Reductions and Order 16
14.3 NP-Hard Languages 17
14.4 NP-Complete languages 18
NP-COMPLETENESS PROOFS 18

Lecture 04: Boolean circuits, Cook's theorem and SAT **19**
RECAP 19

15 Boolean Circuits **20**
15.1 Boolean gates 20
15.2 Boolean Circuit 20
15.3 Truth assignments 21
15.4 Configuration 22

16 Boolean Circuit Satisfiability Problem BC-SAT **22**
16.1 BC-SAT is a NP problem 23

17 Cook's Theorem **24**
17.1 Cook's Reduction 24
17.2 Buinding the circuit 24
17.3 Crucial points: 25

18 Formula satisfiability problem (SAT) **26**
18.1 formula encoding : a tree 27
18.2 Boolean formula truth assignment 27
18.3 Formula SAT problem formulation 27
18.4 Wrong Exponential Reduction 28

Lecture 05: Satisfiability reductions and graph problems **29**
RECAP . . . 29
PROBLEMS ON GRAPHS . . . 33

Lecture 06: Graph problems and SUBSET SUM **34**
RECAP . . . 34

Lecture 07a: The SUBSET SUM reduction **42**
3-CNF-SAT to SUBSET SUM (SS) . . . 42

Lecture 07b: Exercises on complexity and NP-hardness **45**
Exercise: Decision algorithms for NP languages . . . 45
Exercise: The RAM halting problem . . . 45
Exercise: Closure of NP under reductions . . . 45
Exercise: Circuit testing . . . 45
Exercise: KNAPSACK . . . 46
Exercise: Complementary languages . . . 46

Lecture 08: Exercises on reductions and search **46**
Exercise: co-NP and its relationship to NP . . . 46
Exercise: Dominating sets . . . 46
DOMINATING SET (DS) . . . 47
Exercise: LONGEST PATH . . . 47
Exercise: From SUBSET SUM decision to search . . . 47
Exercise: PARTITION . . . 48

Lecture 09: Approximation algorithms **48**
APPROXIMATION ALGORITHMS . . . 48
A TWO-APPROXIMATION ALGORITHM FOR (MINIMUM) VERTEX COVER . . . 50

Advanced Algorithm Design (AAD)

Prof. G. Pucci

	Mondays	14:30–15:15	15:20–16:05	Pe
Classes	Tuesdays	10:30–11:15	11:20–12:05	Pe
	Wednesdays	12:30–13:15	13:20–14:05	Pe

Office Hours (DEI/G, room 414) Wednesdays: 15:30–17:00. Compulsory reservation by (same) wednesday morning via mail: `geppino.pucci@unipd.it`.

MOODLE: registration is compulsory! Follow regularly for announcements, slides, etc.

WEBPAGE: `www.dei.unipd.it/~geppo/AAD`. Surf to get to know the sections.

1 Advanced Algorithm Design (AAD)

First Semester, 2026/27. Laurea Magistrale in Computer Engineering.

- Instructor ← Info about me, office hours.
- Syllabus ← High level description of course topics.
- Textbooks.
- Lectures Diary A.Y. 2026/27: Lecture-by-lecture — This year.
- Lectures Diary A.Y. 2025/26: Lecture-by-lecture — Last year.
- Exam Rules.
- Exam Sessions ← Dates/rooms already fixed!
- Online Material ← Supplementary material.

1.1 Objective:

Integrate and expand the knowledge acquired in the introductory course on algorithms : Algoritmi per l'ingegneria. The material is mainly theoretical but has enormous practical impact.

1.2 CourseTopics

- NP-Completeness: more concrete approach (non determinism — verifiability).
- Reduction Techniques.
- Approximation algorithms for hard problems (solutions of guaranteed quality).
- Computational aspects of number theory (divisibility, modular arithmetic, primality) and cryptographic applications of hardness.
- Randomization: general techniques for designing randomized algorithms.

1.3 Course objective:

Emphasis on rigorous proof techniques and general paradigms but will always discuss the practical relevance of the material.

Textbook and resources:

- CLRS (Introduction to Algorithms 3rd ed): Cormen, Leiserson, Rivest, Stein.
- Abundant supplementary material (see web page).

Attendance Bonus +1 pt for those students attending ≥ 30 lectures.

1.4 Written Exam

Part 1: theory questions on covered material (proofs, definitions, analyses): 30%.

Part 2: creative exercises apply techniques to new problems (lots of exercise sessions during lectures): 70%.

Voluntary teamwork: groups of 2 students. Read and understand advanced material, prepare a short written report plus class-presentation (with slides).

Evaluation: Exemption from Part 1 and 30% of final grade.

- team creation + assignment: first weeks of december.
- presentation must be done in January (last week of lectures), Jan 11–13.

1.5 Pre-requisites

- Asymptotic notation: O, Ω, Θ.
- Summations
- basic probability (distributions, averages, variance, random variables)
- Divide-and-conquer: design (recursion), analysis (recurrences, master theorem).
- Dynamic Programming paradigm
- Greedy paradigm

If these topics are not in your background you should acquire them by yourself (use CLRS: Chaps 3, 4, 15, 16, App A, B, C).

I will be glad to help students with difficulties (use office hours/email).

2 Complexity Classes and Theory of Intractability (NP-Completeness)

The objective is to categorize computational problems according to the complexity of the algorithms for their solution.

2.1 Computational Problem

$$\Pi \subseteq \mathcal{I} \times \mathcal{S} \quad \text{(relation)}, \qquad (i, s) \in \Pi \quad [i \; \Pi \; s] \quad \Longleftrightarrow \quad s \text{ is a solution to } i.$$

The computational problem is a set of instance solution pairs (i, s) such that s is a solution to i.

It is a many to many correspondence: an instance $i \in \mathcal{I}$ may have many solutions $s \in \mathcal{S}$ and a solution $s \in \mathcal{S}$ may be a solution to many instances $i \in \mathcal{I}$.

Algorithm that solves Π implements this function:

$$A_\Pi : \mathcal{I} \longrightarrow \mathcal{S}, \qquad \forall i \in \mathcal{I} : \quad i \; \Pi \; A_\Pi(i) \in \mathcal{S}.$$

2.2 Complexity of a problem

We are interested in upper/lower complexity bounds on algorithms for Π:

$$\underbrace{\exists A_\Pi \text{ solving } \Pi: \ T_A(n) = O(f(n))}_{\text{UPPER BOUND}},$$

$$\underbrace{\forall A_\Pi \text{ solving } \Pi: \ T_A(n) = \Omega(f(n))}_{\text{LOWER BOUND}}.$$

The lower bound is generally more difficult to prove than the upper bound. The order of $f(n)$ captures the difficulty of Π.

2.3 Problem Categories

1. Tractable problems: Problems admitting polynomial algorithms

$$\exists A_\Pi: \quad T_{A_\Pi}(n) = O(n^k), \quad k \geq 0.$$

2. Provably intractable problems : Problems admitting an exponential lower bound

$$\exists c > 1: \ \forall A_\Pi: \quad T_{A_\Pi}(n) = \Omega(c^n).$$

3. Undecidable /Unsolvable problems : Problems not admitting any algorithm
4. Intractable problems: problems for which neither an exponential lower bound nor a polynomial upper bound is known

2.4 HALTING PROBLEM

$$\Pi_{\text{HALT}}: \quad \mathcal{I} = \{\langle M, x\rangle : \ M \text{ Turing Machine}, \ x \in \{0,1\}^*\}, \qquad \mathcal{S} = \{\text{yes}, \text{no}\}.$$
$$\langle M, x\rangle \ \Pi_{\text{HALT}} \text{ yes} \quad \Longleftrightarrow \quad M(x) \text{ halts}.$$

2.5 Intractable Problems

- Problems of huge practical interest (economics, electronics, AI, learning, biology, logistics).
- Strong structural properties: a polynomial algorithm for any single intractable problem yields polynomial algorithms for all intractable problems (via reductions, NP-Completeness theory).

We'll deal with classes of problems: we need to standardize computational problems as much as possible.

3 Abstract Decision Problems

$$\Pi_D \subseteq \mathcal{I} \times \mathcal{S}, \qquad \mathcal{S} = \{\text{yes}, \text{no}\}.$$

Fundamental parts in the formulation of an abstract decision problem:

- description of the generic instance.
- question on the instance.
- the solution is a yes/no answer to the question.

3.1 EXAMPLE: Membership of integer key in a set

$$\begin{cases} \text{INSTANCE: } \langle S, m\rangle : \ S \subseteq \mathbb{N}, \text{ finite}, \ m \in \mathbb{N}, \\ \text{QUESTION: } m \in S? \end{cases}$$

A decision problem is a function

$$\Pi_D : \mathcal{I} \longrightarrow \{\text{yes}, \text{no}\}.$$

Yes/No instances are also called Positive/negative instances.

3.2 Restriction to decision problems: excessive?

We restrict our analysis on computational problems to decision problems. This is not a severe restriction since many optimization problems can be reduced to decision problems. In practice we usually solve more complex optimization problems.

4 Optimization Problem

$$\Pi_{\text{opt}} \subseteq \mathcal{I} \times \mathcal{S}, \qquad c : \mathcal{S} \longrightarrow \mathbb{N} \cup \{+\infty\}.$$

$\forall i \in \mathcal{I}$: determine $s^* \in \mathcal{S}(i) = \{s : i \ \Pi \ s\}$ (feasible solutions) such that:

$$c(s^*) = \min/\max\{c(s) : s \in \mathcal{S}(i)\}.$$

4.1 Example : Shortest Unweighted Path Problem (SUP)

$$\mathcal{I} = \{\langle G = (V, E), u, z\rangle : \ V \subseteq \mathbb{N}, \text{ finite}, \ E \subseteq V \times V, \ u, z \in V\}.$$

$\mathcal{S} = \mathbb{N}^*$ (sequences of naturals: paths).

- $i \ \Pi \ s$ if

$$(s = \langle v_1, v_2, \ldots, v_k\rangle, \ v_i \in V) \ \wedge \ (v_1 = u, \ v_k = z) \ \wedge \ (v_i, v_{i+1}) \in E, \quad 1 \le i < k,$$

  where $i = \langle G = (V, E), u, z\rangle$.
- $i \ \Pi \ \varepsilon$ if $\nexists$ path from u to z in G.

$$c(\langle v_1, \ldots, v_k\rangle) = k - 1, \quad k \ge 1; \qquad c(\varepsilon) = +\infty.$$

This is a minimization problem.

An optimization problem can be easily associated to a decision problem using the following standard technique: augment instance with a natural value representing an upper (MIN) or lower (MAX) limitation to the objective function, for all feasible solutions.

4.2 Example : K Unweighted Path Problem (KUP)

$$\begin{aligned} &\text{I: } \langle G = (V, E), u, z, k\rangle : \ V \subseteq \mathbb{N}, \text{ finite}, \ E \subseteq V \times V, \\ &\quad u, z \in V, \ k \in \mathbb{N}. \\ &\text{D: } \exists \text{ path from } u \text{ to } z \text{ of length } \le k? \end{aligned}$$

The decision problem is not fully equivalent but it holds that: A polynomial algorithm for KUP can be used as a subroutine to solve SUP in polynomial time

Let $A_{\text{KUP}}(G = (V, E), u, z, k)$ be a polynomial time algorithm for KUP.

We first show that A_{KUP} can be employed to determine the length of the shortest path between u and z as follows:

P-LENGTH$(\langle G = (V, E), u, z\rangle)$

```
n ← |V|
for k ← 0 to n − 1 do
   if A_KUP(⟨G = (V, E), u, z, k⟩) = yes
      then return k
return +∞
```

Clearly, P-LENGTH(G, u, z) returns the smallest value of k such that $\exists$ a path from u to z ($+\infty$ if no path exists).

Since $|V| = n \leq |\langle G, u, z\rangle|$ (SIZE OF THE INSTANCE) and

$$|\langle G, u, z, k\rangle| = \Theta(|\langle G, u, z\rangle|),$$

if $T_{\text{KUP}}(\langle G, u, z, k\rangle) = O(|\langle G, u, z, k\rangle|^c)$ then

$$T_{\text{P-LENGTH}}(\langle G, u, z\rangle) = O(|\langle G, u, z\rangle|^{c+1}).$$

Once the length k^* of the shortest path is established, it is easy to determine the actual path through $|E|$ calls to A_{KUP}.

Algorithm idea: progressively eliminate edges which are not necessary to guarantee the existence of a shortest path. To check this, we use A_{KUP} as an ORACLE.

$A_{\text{SUP}}(\langle G = (V, E), u, z\rangle)$

```
k* ← P-LENGTH(⟨G, u, z⟩) {see above}
if (k* = +∞) then return "no path"
* E = {e_1, e_2, ..., e_|E|} *
E' ← E
for i ← 1 to |E| do
   if A_KUP(⟨G' = (V, E' − {e_i}), u, z, k*⟩) = yes
      then E' ← E' − {e_i}
   {∃ shortest path π* ∈ E' not using e_i}

* E' contains only the edges of a s.p.!! *

return π*
```

Correctness: if $k^* = +\infty$, A_{SUP} is correct. Otherwise, by contradiction, if after the for loop, E' does not contain only the edges of a single shortest path π^*:

$$\exists e_{\bar{i}} \in E' \text{ in } E' \text{ not used by } \pi^*.$$

BUT at iteration $\bar{i}$ we would have:

$$A_{\text{KUP}}(\langle G' = (V, E' - \{e_{\bar{i}}\}), u, z, k^*\rangle) = \text{yes!}$$

Thus $e = e_{\bar{i}}$ would have been removed!

$\rightarrow$ We have obtained a poly-time algorithm A_{SUP} for SUP!

(EXERCISE: determine complexity)

Morale: restriction to decision problems is without loss of generality (w.l.o.g.) if we are only interested in determining existence of poly-time algorithms.

5 Lesson 1 Recap

- Problem classes are categorization of problems with respect to complexity of the algorithms that solve them : they allow standardization.
- Abstract decision problems: Problems whose solution set is 1,0 $\Pi_A : \mathcal{I} \longrightarrow \{\text{yes}, \text{no}\}$.
- Optimization problems can be paired with a computationally equivalent decision problem, with an upper/lower bound on cost added to the instance.
- If decision problem admits a poly-time algorithm then same holds for optimization problem. Proof : a simple for loop on the cost of the optimization problem, using the decision problem as a subroutine.

There are some optimization problems where the equivalence does not hold, but it holds for the totality of the problems that we are interested in.

Morale: We can concentrate on decision problems.

6 Classes of decision problems

- Abstract decision problems: $\Pi_A : \mathcal{I} \longrightarrow \{\text{yes}, \text{no}\}$.
- Concrete decision problems: $\Pi_C : \{0,1\}^* \longrightarrow \{\text{yes}, \text{no}\}$.

Problems are elements of sets (classes). There is a need of standard format. Decision problems's solution sets $\mathcal{S}$ are already standardized:

$$\mathcal{S} = \{\text{yes}, \text{no}\}, \qquad \Pi_A : \mathcal{I} \longrightarrow \{\text{yes}, \text{no}\}.$$

There is a need to standardize $\mathcal{I}$: We can transform any instance set from a set of abstract structures of arbitrary nature to a set of binary strings $\subseteq \{0,1\}^*$

7 Encodings

An encoding is an injective function, mapping an abstract set $\mathcal{I}$ to $\{0,1\}^*$:

$$e : \mathcal{I} \longrightarrow \{0,1\}^*.$$

An encoding must also be coincise, i.e. the length of the encoding must be polynomially bounded in the size of the abstract instance. (Explained later.)

An encoding turns an abstract decision problem Π_A into a concrete decision problem:

$$\begin{array}{rcl} \Pi_A : \mathcal{I} & \longrightarrow & \{0,1\} \\ \downarrow e(\cdot) & & \\ \Pi_C : \{0,1\}^* & \longrightarrow & \{0,1\}. \end{array}$$

The two problems are related as follows:

$$\forall x \in \{0,1\}^* : \quad (1) \quad \Pi_C(x) = 1 \Longleftrightarrow \exists i \in \mathcal{I} : (x = e(i)) \wedge (\Pi_A(i) = \text{yes}).$$

NOTE 1: $\Pi_C(x)$ is defined on all strings in $\{0,1\}^*$. From $(1, \Rightarrow)$ we have:

$$\forall x \in \{0,1\}^* : \quad \nexists i \in \mathcal{I} : x = e(i) \;\Rightarrow\; \Pi_C(x) = 0.$$

NOTE 2: The size of a concrete instance is $|x|$

An algorithm A_{Π_C} for a concrete problem takes a binary string as input and outputs 0 or 1.

7.1 Encoding Choice

The choice of the encoding is a delicate issue with critical impact on the complexity analysis of the algorithm.

$$T(n) = \max\{\text{cost of instructions executed by } A_\Pi(i), \text{ with size}(i) = n\}.$$

(Standard cost model: each instruction costs 1.)

Given an algorithm A_{Π_C} for a concrete problem Π_C:

$$T_{A_{\Pi_C}}(n) = \max_{|x|=n}\{\#\text{ instructions executed by } A_{\Pi_C}(x)\}.$$

Since $|x| = |e(i)| = n$ (i: abstract instance), the independent parameter is the encoding length

"Unreasonably" lengthy encodings may "mask" the inefficiency of an algorithm

7.2 Binary Encoding

Example : let $\mathcal{I} = \mathbb{N} - \{0\}$.

If $m \in \mathcal{I} = \mathbb{N} - \{0\}$,

$$e_b(m) = \langle a_k, a_{k-1}, \ldots, a_0\rangle, \quad a_k = 1,$$

with $k = \lfloor \log_2 m \rfloor$ and

$$m = \sum_{j=0}^{k} a_j 2^j.$$

NOTE:

$$|e_b(m)| = \lfloor \log_2 m \rfloor + 1 = \Theta(\log m).$$

7.3 Unary Encoding

If $m \in \mathcal{I} = \mathbb{N} - \{0\}$,

$$e_u(m) = \langle \underbrace{1, \ldots, 1}_{m \text{ times}} \rangle \quad \text{(string of } m \text{ 1's)}.$$

NOTE: $|e_u(m)| = m$.

NOTE:

$$m = |e_u(m)| = 2^{\log_2 m} = \Theta(2^{|e_b(m)|}).$$

The unary encoding is exponentially longer than the binary encoding!

7.4 Ripercussions on complexity analysis

$T(n)$ uses encoding length as the independent variable n.

Consider this algorithm:

GAUSS(m)

```
sum ← 0
for i ← 1 to m do sum ← sum + i
return sum
```

GAUSS computes $\sum_{i=0}^{m} i = m(m+1)/2$ executing m iterations. But

$$m = |e_u(m)| = \Theta(2^{|e_b(m)|}).$$

$\Rightarrow$

$$T^U_{\text{GAUSS}}(\underbrace{|e_u(m)|}_{n}) = \underbrace{|e_u(m)|}_{n} \quad \text{(independent parameter)}.$$

GAUSS takes linear time in the size of the input under the UNARY encoding!

BUT

$$T^B_{\text{GAUSS}}(\underbrace{|e_b(m)|}_{n'}) = m = \Theta(\underbrace{2^{|e_b(m)|}}_{2^{n'}}).$$

GAUSS takes exponential time in the size of the input under the BINARY encoding!

The same algorithm is associated to drastically different complexities by the two encodings.

Is GAUSS good or bad? It's bad! For $m = 2^{32}$ GAUSS executes $m \simeq 4$ billion sums.

while

GAUSS2(m)

return $m \cdot (m+1)/2$

executes 3 operations between numbers of size ≤ 64.

MORALE: The linear complexity under the unary encoding is MISLEADING. The fact that GAUSS is bad is masked by the sheer length of the unary encoding.

We will use only concise encodings whose length is comparable (i.e., at most polynomially larger) with a minimum-length encoding.

7.5 Coincise Encodings

From now on we will ignore the details of the encoding and assume that a concise encoding is in use.

- Naturals must be encoded to the base $b \geq 2$, $b = O(1)$.
- A set of naturals is encoded by concatenating the encodings of its elements.

NOTATION: Given a set of abstract instances $\mathcal{I}$: $\forall i \in \mathcal{I}$, $\langle i \rangle \in \{0,1\}^*$ denotes a concise encoding of i. e.g., $\langle G = (V, E) \rangle$.

Standardization is now complete. We can now standardize our analysis using concrete decision problems:

$$\Pi_C : \{0,1\}^* \longrightarrow \{0,1\}.$$

ANY concrete decision problem maps binary strings (instance encodings) into bits (1/0 encoding yes/no). We can map any problem into a concrete decision problem, and analyze the concrete decision problem instead of the original problem. The results of the analysis will be valid for the original problem as well.

8 Formal Language Representation of Concrete Decision Problems

Given an alphabet Σ, a formal language L over Σ is a set of finite strings:

$$L \subseteq \Sigma^* \quad \text{(Kleene star of } \Sigma\text{)}.$$

- L may/may not contain the empty string ε ($\varepsilon \notin L$ or $\varepsilon \in L$).
- Empty language: $\varnothing$ ($\{\varepsilon\} \neq \varnothing$!) is a language with no strings.
- Union operator: $L_1 \cup L_2 = \{z \in \Sigma^* : \exists x \in L_1, y \in L_2 : z = x \text{ or } z = y\}$. The union of two languages is a language where each string is in at least one of the two languages.
- Intersection operator: $L_1 \cap L_2 = \{z \in \Sigma^* : \exists x \in L_1, y \in L_2 : z = x \text{ and } z = y\}$. The intersection of two languages is a language where each string was in both original languages.
- $L_1 \cdot L_2$ concatenation operator:

  $$L_1 \cdot L_2 = \{z \in \Sigma^* : \exists x \in L_1, y \in L_2 : z = \langle x, y \rangle\}.$$

  The concatenation of two languages is a language where each string is the concatenation of a string from the first language and a string from the second language.
- Powers: $L^0 = \{\varepsilon\}$; $L^i = L \cdot L^{i-1}$. The power of a language is a language where each string is the concatenation of i strings from the original language.
- Kleene star of a language:

  $$L^* = \bigcup_{i=0}^{\infty} L^i = \{z \in \Sigma^* : \exists k \geq 1 : \exists x_1, \ldots, x_k \in L : z = \langle x_1, x_2, \ldots, x_k \rangle\} \cup \{\varepsilon\}.$$

  The kleene star of a language is a language where each string is the concatenation of any number of strings from the original language, including zero strings (the empty string).

8.1 Formal languages and concrete decision problems

There is a one-to-one correspondence between concrete problems and formal languages over $\Sigma = \{0, 1\}$:

$$\Pi_C \longleftrightarrow L_{\Pi_C} = \{x \in \{0,1\}^* : \Pi_C(x) = 1\} \quad \text{(encodings of positive instances)}.$$

We will use the two concepts interchangeably.

$$\text{Problem } \Pi_C \longleftrightarrow \text{Algorithm solving } \Pi_C,$$
$$\text{Language } L_{\Pi_C} \longleftrightarrow \text{Algorithm deciding } L_{\Pi_C} \text{ (Automaton)}.$$

DEF:

$$L_A = \{x \in \{0,1\}^* : \ A(x) = 1\} \quad \text{(language decided by } A\text{)}.$$

- An automaton A decides a language L if it accepts all strings in L and rejects all strings not in L.
- An algorithm that solves a concrete decision problem Π_C is an algorithm that decides the language L_{Π_C}. It returns 1 for all strings in L_{Π_C} and 0 for all strings not in L_{Π_C}.
- Algorithm A accepts x if $A(x) = 1$. A rejects x if $A(x) = 0$.

8.2 Polynomial-time decision problems and the complexity class P

A language L is decided in polynomial time if $\exists A$:

1. $L = L_A$.
2. $\exists k \in \mathbb{N} : \ \forall x \in \{0,1\}^* : \ T_A(|x|) = O(|x|^k)$ (A runs in polynomial time).

Equivalently, a concrete decision problem Π_C is solved in polynomial time if $\exists A$:

1. $\forall x \in \{0,1\}^* : \ A(x) = \Pi_C(x)$.
2. $\exists k \in \mathbb{N} : \ \forall x \in \{0,1\}^* : \ T_A(|x|) = O(|x|^k)$ (A runs in polynomial time).

8.2.1 DEF complexity class:

A complexity class is a set of languages .(equivalently: a set of concrete decision problems).

8.2.2 DEF class P:

$$\mathrm{P} = \{L \subseteq \{0,1\}^* : \ L \text{ is decided in polynomial time}\}.$$

Alternatively

$$\mathrm{P} = \{\Pi_C : \{0,1\}^* \to \{0,1\} : \ \Pi_C \text{ is solved in polynomial time}\}.$$

8.3 Search Problem

The Search problem is a problem that asks whether a given element m is contained in a given finite set $S \subseteq \mathbb{N}$.

It can be formalized as a concrete decision problem L_{SEARCH}.

$$\begin{aligned} &\text{INSTANCE: } \langle S, m\rangle : \ S \subseteq \mathbb{N} \text{ finite}, \ m \in \mathbb{N}, \\ &\text{QUESTION: } m \in S? \end{aligned}$$

We can write the relative formal language L_{SEARCH} as follows:

$$L_{\text{SEARCH}} = \{x \in \{0,1\}^* : \ (x = \langle S, m\rangle) \wedge m \in S\}.$$

Where x is a binary string, and x is in the language only if it is the encoding of a pair $\langle S, m\rangle$ where m is inside S.

We can solve the search problem with a simple algorithm A_{SEARCH} that first checks if the input is a valid encoding, and then iterates through the elements of S to check if m is present. If m is found, it returns 1 (yes), otherwise it returns 0 (no). $A_{\text{SEARCH}}(x)$

```
if (x ≠ ⟨S, m⟩) then return 0
k ← |S|
* S = {s_1, ..., s_k} *
for i ← 1 to k do
    if (s_i = m) then return 1
return 0
```

Since we expect x to be a concise encoding, we can assume that the size of the set S is $|S| = O(|x|)$. Therefore, the algorithm runs in linear time with respect to the size of the input string x:

$$T_{A_{\text{SEARCH}}}(|x|) = O(|x|) \ \Rightarrow \ \{\text{SEARCH}, L_{\text{SEARCH}}\} \in \mathrm{P}.$$

9 Polynomial time verification

We need to remember that Solving a problem means building a solution from scratch, while Verifying a solution means checking the correctness of a given candidate solution.

Example: Recurrences solving: build the closed formula. verifying: given a candidate closed formula check its correctness.

Example: Formal languages solving: create an algorithm that decides if $x \in L$. verifying: two-input algorithm: x plus "additional info" y. Algorithm checks that y certifies that $x \in L$ (e.g., $y =$ closed formula).

Verifying is easier than solving! (Or is it?)

10 Polynomial-time verification of Hamiltonian cycles

Given $G = (V, E)$ undirected, a simple cycle is a sequence:

$$\langle v_1, v_2, \ldots, v_k \rangle, \quad v_i \in V, \quad v_1 = v_k,$$

$$v_i \neq v_j \quad 1 \leq i < j < k, \qquad k \geq 1, \qquad \{v_i, v_{i+1}\} \in E \quad 1 \leq i < k.$$

HAMILTON problem is a decision problem that asks whether a given undirected graph contains a simple cycle that visits all nodes in the graph exactly once (a Hamiltonian cycle):

I: $\langle G = (V, E) \rangle$, $\quad G$ undirected,

D: is there in G a simple cycle containing all nodes in V?

Example:



10.1 Solution

Deciding whether a graph contains a Hamiltonian cycle is a difficult problem (no polynomial algorithms are known).

10.2 Verification

Verifying that a candidate vertex sequence is indeed a Hamiltonian cycle is easy (there is a polynomial time algorithm that checks the correctness of a candidate solution). Given a candidate sequence $\langle v_1, v_2, \ldots, v_k \rangle$, we can check in polynomial time whether it is a simple cycle and whether it visits all nodes in the graph exactly once. If both conditions are satisfied, we can conclude that the graph contains a Hamiltonian cycle.

VERIFY-HAMILTON(x, y)

if $(x \neq \langle G = (V, E)\rangle)$ then return 0
if $(y \neq \langle v_1, \ldots, v_{|V|+1}\rangle,\ v_i \in V)$ then return 0
{check that input is "well formed"}
if $(v_1 \neq v_{|V|+1})$ then return 0
{candidate cannot be a cycle!}
if $(\exists i, j :\ 1 \leq i < j \leq |V| :\ v_i = v_j)$ then return 0
{check that y contains all nodes}
for $i \leftarrow 1$ to $|V|$ do
  if $(\{v_i, v_{i+1}\} \notin E)$ then return 0
{check cycle edges}
return 1

The algorithm is at most quadratic in the size of the input! So we have a polinomial time verification algorithm for the hamiltonian cycle problem. The algorithm runs in polynomial time in the size of the input x and the size of the certificate y, so we have that HAMILTON $\in$ NP.

DEF: A two-input algorithm V verifies string $x \in \{0,1\}^*$ iff $\exists y \in \{0,1\}^* :\ V(x, y) = 1$. Any $y :\ V(x, y) = 1$ is called a certificate for x.

For instance, if $x = \langle G = (V, E)\rangle \in L_{\text{HAMILTON}}$, a certificate for x is a sequence of vertices that forms a Hamiltonian cycle in G:

$$\begin{aligned} &y = \langle v_1, \ldots, v_{|V|+1}\rangle, \quad v_1 = v_{|V|+1}, \\ &v_i \neq v_j \quad 1 \leq i < j \leq |V|, \qquad (v_i, v_{i+1}) \in E. \end{aligned}$$

IMPORTANT: if $x \notin L$ then no certificate may exist!

The language verified by a two-input algorithm V is a language that contains all encodings of instances that have a certificate (so all encodings of graphs that have a Hamiltonian cycle):

$$\begin{aligned} L_V &= \{x \in \{0,1\}^* :\ V \text{ verifies } x\} \\ &= \{x \in \{0,1\}^* : \exists y \in \{0,1\}^* :\ V(x, y) = 1\}. \end{aligned}$$

OBSERVATION: VERIFY-HAMILTON is a verification algorithm for L_{HAMILTON} (or, equivalently, for HAMILTON).

We only require that one certificate exists but do not ask the algorithm to compute it itself.

The certificate is provided to the algorithm externally. "Computing the certificate" would imply solving the problem.

A language L is verified in polynomial time if and only if there exists a two input algorithm $V = (x, y)$ that verifies that y is a certificate for x in polynomial time in the size of the input x and the size of the certificate y for all $x \in L$, where y is a certificate for x that must be polynomial in the size of the input x.

L is verified in polynomial time iff $\exists V$:

1. $L = L_V$.
2. $\exists k_1 \geq 0 :\ \forall x \in L\ \exists y :\ V(x, y) = 1$ and $|y| = O(|x|^{k_1})$.
3. $\exists k_2 \geq 0 :\ T_V(|x|, |y|) = O((|x| + |y|)^{k_2})$.

The certificate y must also be of polynomial size in the size of the input x otherwise the complexity would be dominated by the size of the certificate and not by the size of the input.

We need SHORT CERTIFICATES!

10.3 Class NP and the relationship between P and NP

The complexity class NP is the set of all languages that are verified in polynomial time.

$$\text{NP} = \{L \subseteq \{0,1\}^* : \ L \text{ verified in polynomial time}\}.$$

NP can be equivalently defined with respect to decision problems :

$$\text{NP} = \{\Pi_C : \{0,1\}^* \to \{0,1\} : \ \Pi_C \text{ is verified in polynomial time}\}.$$

We proved that HAMILTON $\in$ NP.

Since solving in polynomial time implies verifying in polynomial time (proof later), we have that

$$\text{if } L \in \text{P} \ \Rightarrow \ L \in \text{NP}.$$

That is, P $\subseteq$ NP.

But we do not know if P = NP or P $\subset$ NP.

11 Lesson 2 Recap

Encodings: $\Pi_A \xrightarrow{\mathcal{I} \to e(\mathcal{I})} \Pi_C : \{0,1\}^* \to \{0,1\}$. We want to have reasonable/concise encodings.

Formal languages and decision problems are interchangeable:

$$\underbrace{\Pi_C}_{\text{solution}} \longleftrightarrow \underbrace{L_{\Pi_C} = \{x \in \{0,1\}^* : \Pi_C(x) = 1\}}_{\text{decision}}.$$

The solution of the decision problem corresponds to the decision of the formal language and vice versa.

Poly-time solution/decision is obtainable: the problem/language is in P.

Verification: algorithm $V(x, y)$.

$$L_V = \{x \in \{0,1\}^* : \exists y \in \{0,1\}^* : \ V(x,y) = 1\}.$$

Poly-time verification algorithm exists: the problem/language is in NP. Beware of the constraints :

1. poly-size certificate $[|y| = O(|x|^{k_1})]$;
2. poly-time in both inputs $[T_V(x,y) = O((|x| + |y|)^{k_2})]$.

12 Theorem and proof: P is a subset of NP

P $\subseteq$ NP ($L \in \text{P} \Rightarrow L \in \text{NP}$).

Proof: Let $L \in$ P. We'll prove $L \in$ NP by constructing a poly-time verifier: Recall that $L \in$ P implies that $\exists A_L$ which decides L in time $T_{A_L}(|x|) = O(|x|^k)$.

We can build a 2 input verifier $V_L(x, y)$ uses the algorithm A_L as a subroutine:

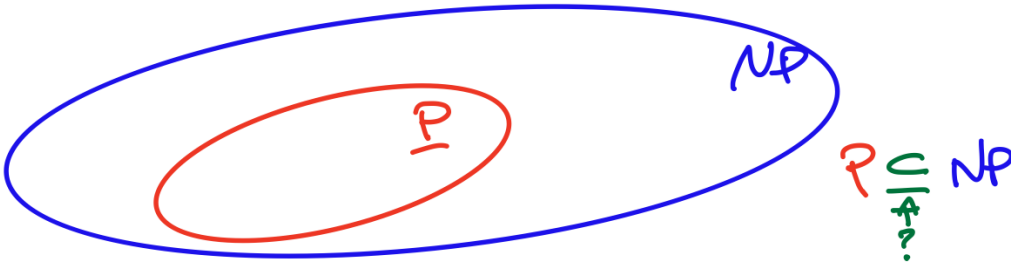
$V_L(x, y)$

return $A_L(x)$

The verifier disregards y and calls $A_L(x)$.

We can prove that V_L is a poly-time verifier for L:

1. $L_V = L$ ($V_L(x, y) = 1 \iff x \in L$).
2. $y = \varepsilon$ is a certificate $\forall x \in L$
3. y is polynomial in $|x|$: $|\varepsilon| = O(|x|^0)$: $k_1 = 0$
4. $T_V(|x|, |y|) = T_{A_L}(|x|) = O(|x|^k) = O((|x| + |y|)^k)$ $(k_2 = k)$.



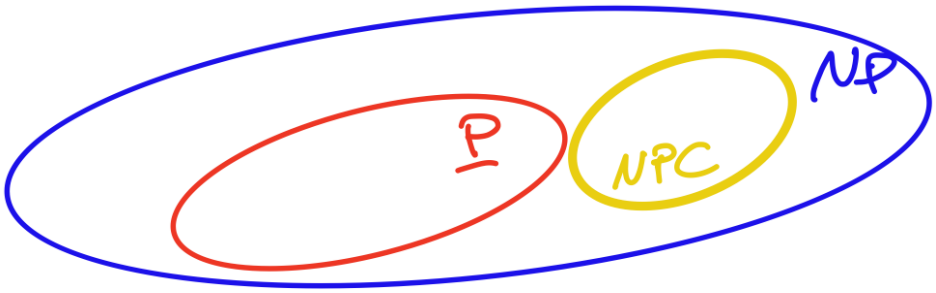
What can we say about NP − P? Could be empty! (P = NP) Could be nonempty! (P ≠ NP)

The question P = NP? is still open (1 million-dollar question).

12.1 Reasonable guess and evidence that P is not NP:

P ≠ NP.

1. Verifying seems intrinsically easier than solving.
2. We'll define a partial order in NP (based on solution difficulty). For the most difficult problems in NP no poly-time algorithms are known.
3. The most difficult problems in NP form an "equivalence class": NPC (NP-complete problems).



$$\text{If P} \neq \text{NP} \Rightarrow \text{NPC} \subset \text{NP} - \text{P} \quad \text{(LIKELY!)}$$
$$\text{If P} = \text{NP} \Rightarrow \text{NPC} \subset \text{P} \quad \text{(UNLIKELY!)}$$

We will see that:

- NPC contains many decision problems from different fields of huge practical interest.
- Problems in NPC are "equivalent" in the following sense:

A poly-time solution to any one problem in NPC immediately yields a poly-time solution to all problems in NPC

13 Reducibility between problems

Intuitively, Π_1 can be reduced to Π_2 if any instance i_1 of Π_1 can be 'easily' transformed into an instance i_2 of Π_2 so that a solution to i_2 provides a solution to i_1.

For decision problems:

$$i_1 \longmapsto f(i_1) = i_2, \qquad \Pi_1(i_1) = \Pi_2(i_2).$$

The map function f must transform positive instances into positive instances and negative instances into negative instances.

13.1 Example: reducibility between first and second degree equations

$$\Pi_1: \quad \begin{cases} \text{I: } \langle a, b, x\rangle, \quad a, b, x \in \mathbb{C}, \\ \text{Q: } ax + b = 0? \end{cases}$$

(decision version of first degree equations)

$$\Pi_2: \quad \begin{cases} \text{I: } \langle c, e, f, x\rangle, \quad c, e, f, x \in \mathbb{C}, \\ \text{Q: } cx^2 + ex + f = 0? \end{cases}$$

(decision version of second degree equations)

We can reduce Π_1 to Π_2 as follows

$$i_1 = \langle a, b, x\rangle \xrightarrow{f} \langle \underbrace{0}_{c}, \underbrace{a}_{e}, \underbrace{b}_{f}, x\rangle = i_2.$$

Clearly, $\Pi_1(i_1) = \Pi_2(f(i_1)) = \Pi_2(i_2)$.

$$0 \cdot x^2 + ax + b = ax + b = 0?$$

14 Order between Problems

NOTE: An "efficient" reduction from Π_1 to Π_2 implies that Π_1 is "less difficult" than Π_2.

(If I can solve Π_2 efficiently, then I can also solve Π_1 efficiently.)

"Efficient" reducibility creates an order between problems.

14.1 Polynomial time reducibility

DEF: A function $f : \{0,1\}^* \to \{0,1\}^*$ is polynomial-time computable (ptc) if $\exists$ algorithm $A_f : \{0,1\}^* \to \{0,1\}^*$:

$$\forall x \in \{0,1\}^* : \ A_f(x) = f(x) \text{ and } T_{A_f}(|x|) = O(|x|^k)$$

for some constant $k \geq 0$.

DEF: L_1 is poly-time reducible to L_2 (notation: $L_1 \leq_p L_2$) if:

$$\exists f : \{0,1\}^* \to \{0,1\}^* \text{ ptc}: \qquad x \in L_1 \iff f(x) \in L_2.$$

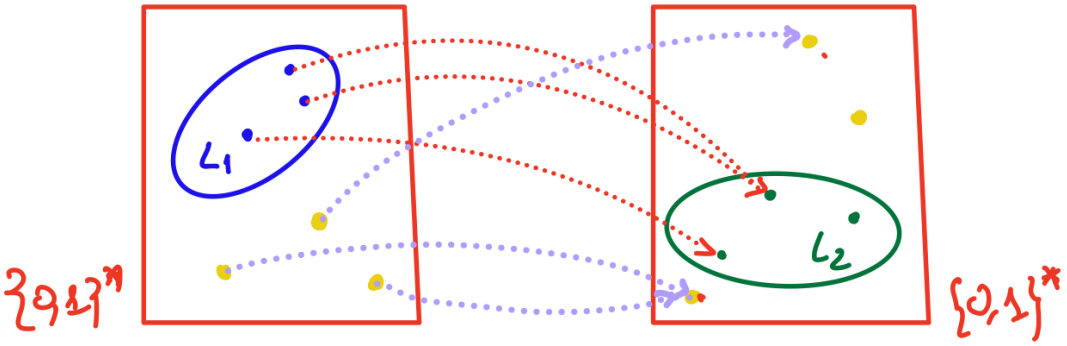
(w.r.t. problems: $\Pi_{L_1}(x) = \Pi_{L_2}(f(x))$)

So if a polinomial time function f exists that transforms instances of Π_1 into instances of Π_2 such that the answer is preserved, then we say that Π_1 is poly-time reducible to Π_2.

$\iff$ means that we need to prove two implications:

- strings in L_1 are mapped to strings in L_2 ($\Rightarrow$);
- strings not in L_1 are mapped to strings not in L_2 ($\Leftarrow \equiv \not\Rightarrow$: COUNTERPOSITIVE).

Remark: f need not be neither injective or surjective.



Poly-time reducibility captures the concept of "lesser difficulty" of L_1 with respect to L_2 in the following sense: $L_1 \leq_p L_2$.

L_1 is at most as difficult as L_2 but it cannot be more difficult than L_2 (otherwise, we would not be able to reduce L_1 to L_2). This is a way to put an order between problems.

We can now study L_2 to get information about L_1. If L_2 can be solved in polynomial time, then L_1 must be solvable in polynomial time as well. If L_2 is not solvable in polynomial time, then we cannot conclude anything about L_1.

14.2 Reductions and Order

Theorem: Given $L_1, L_2 \subseteq \{0,1\}^*$:

$$(L_1 \leq_p L_2) \wedge (L_2 \in \mathrm{P}) \;\Rightarrow\; L_1 \in \mathrm{P}.$$

If L_1 is reducible to L_2 and L_2 can be decided in poly-time, then L_1 can also be decided in poly-time

Proof: We have two hypotheses:

HP1: L_1 is poly-time reducible to L_2: $L_1 \leq_p L_2$: $\exists f$ polynomially time computable (ptc) : $x \in L_1 \iff f(x) \in L_2$.

So we hypothesize that an algorithm A_f exists that computes f in polynomial time. Let $T_{A_f}(|x|) = O(|x|^{k_1})$, $k_1 \geq 0$.

HP2: $L_2 \in \mathrm{P}$: $\exists A_{L_2}$ which decides L_2 in poly time. Let

L_2 is in P, so there exists an algorithm A_{L_2} which decides L_2 in polynomial time. Let$T_{A_{L_2}}(|x|) = O(|x|^{k_2})$.

We can unite these two hypothesis into the following algorithm:

$A_{L_1}(x)$

$y \leftarrow A_f(x)$
return $A_{L_2}(y)$

So to solve instances of L_1, we first compute $y = f(x)$ and then we decide if $y \in L_2$. Since both f and A_{L_2} are polynomial time computable, we can compute the running time of A_{L_1}:

Running time:

$$T_{A_{L_1}}(|x|) = O(|x|^{k_1} + |y|^{k_2}).$$

How large can y be? A_f may use all of its steps to create bits of y, thus

$$|y| = O(|x|^{k_1}).$$

Hence

$$T_{A_{L_1}}(|x|) = O(|x|^{k_1} + |x|^{k_1 k_2}) = O(|x|^{k_3}), \quad \text{where } k_3 = \max\{k_1, k_1 k_2\}.$$

Which is still polynomial in $|x|$.

But so far we only created an algorithm. We need to prove that the algorithm actually decides L_1 correctly. We need to prove that $L_1 = L_{A_{L_1}}$. We can do it by proving both that ($L_1 \subseteq L_{A_{L_1}}$ and $L_{A_{L_1}} \subseteq L_1$):

$$\begin{aligned} x \in L_1 &\Rightarrow y = f(x) \in L_2 \Rightarrow A_{L_1}(x) = A_{L_2}(y) = 1 \\ &\Rightarrow x \in L_{A_{L_1}} \qquad L_1 \subseteq L_{A_{L_1}}. \\ x \in L_{A_{L_1}} &\Rightarrow A_{L_1}(x) = A_{L_2}(y) = A_{L_2}(f(x)) = 1 \\ &\Rightarrow f(x) \in L_2 \Rightarrow x \in L_1 \qquad L_{A_{L_1}} \subseteq L_1. \end{aligned}$$

We proved $L_1 \subseteq L_{A_{L_1}}$ and $L_1 \supseteq L_{A_{L_1}} \Rightarrow L_1 = L_{A_{L_1}}$.

A_{L_1} is a poly-time algorithm deciding L_1, therefore $L_1 \in \mathrm{P}$.

REMARK: The proof uses both implications $x \in L_1 \Longleftrightarrow f(x) \in L_2$.

Poly-time reducibility $\leq_p$ is a relation between languages.

Homework! Prove that:

$$\underbrace{L \leq_p L}_{\text{reflexivity}}, \qquad \underbrace{(L_1 \leq_p L_2) \wedge (L_2 \leq_p L_3) \Rightarrow (L_1 \leq_p L_3)}_{\text{transitivity}}.$$

14.3 NP-Hard Languages

A language is NP-Hard if all languages in NP can be reduced to it in polynomial time.

$L \subseteq \{0,1\}^*$ is NP-hard if

$$\forall L' \in \mathrm{NP}: \quad L' \leq_p L.$$

$$\mathrm{NPH} = \{L \in \{0,1\}^* : \ L \text{ NP-hard}\}.$$

An NP-hard language/problem L is "more difficult" than any problem in NP, in the sense that a poly-time algorithm for L can be transformed into a poly-time algorithm for any problem in NP.

Note: An NP-hard language L need not be in NP itself!

14.4 NP-Complete languages

A language is NP-Complete if it is both in NP and NP-Hard.

DEF: $L \subseteq \{0,1\}^*$ is NP-Complete if

1. $L \in \text{NP}$.
2. $L \in \text{NPH}$.

$$\text{NPC} = \{L \subseteq \{0,1\}^* : \ L \text{ NP-complete}\}.$$

Note: NPC Languages form an overset of NP, $\text{NPC} \subseteq \text{NP}$.

NP-complete problems are the "hardest" problems in NP! In fact, they are all somewhat equivalent, since for all languages in NPC, we can reduce any one of them to any other in polynomial time (this comes from the fact that both are in NP).

$$\forall L_1, L_2 \in \text{NPC}: \quad (\underbrace{L_1}_{\in \text{NP}} \leq_p \underbrace{L_2}_{\in \text{NPC}}) \wedge (\underbrace{L_2}_{\in \text{NP}} \leq_p \underbrace{L_1}_{\in \text{NPC}}).$$

This means that if we can solve one NP-complete problem in polynomial time, then we can solve all NP-complete problems in polynomial time.

Note : If there was a language that was both NP-complete and in P, then all problems in NP would be solvable in polynomial time.

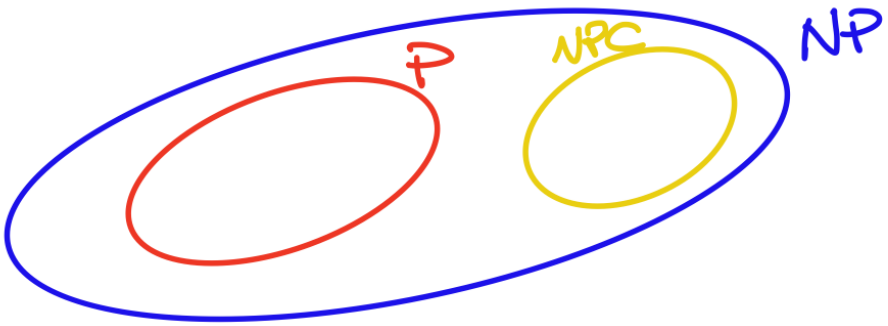
$$\text{If } \exists \bar{L} \in (\text{NPC} \cap \text{P}) \ \Rightarrow \ \text{P} = \text{NP}.$$

Proof: Since $\bar{L} \in \text{NPC}$, we can reduce any languadge $L \in \text{NP}$ to $\bar{L}$ in polynomial time. Since $\bar{L} \in \text{P}$, we can solve $\bar{L}$ in polynomial time. Thus, we can solve any language $L \in \text{NP}$ in polynomial time. $\forall L \in \text{NP}$:

$$(L \leq_p \bar{L}) \wedge (\bar{L} \in \text{P}) \overset{\text{Theorem}}{\Longrightarrow} L \in \text{P}.$$

Thus, $\text{NP} \subseteq \text{P}$, but also $\text{P} \subseteq \text{NP} \Rightarrow \text{P} = \text{NP}$.

No poly-time algorithms are known for NP-complete problems. Evidence points towards $\text{P} \cap \text{NPC} = \varnothing$. If this is the case:



We will often prove results under the hypothesis that $\text{P} \neq \text{NP}$ (or, equivalently, $\text{P} \cap \text{NPC} = \varnothing$).

NP-COMPLETENESS PROOFS

We have defined NPC but how do we place problems in NPC?

Given a candidate language $\bar{L}$:

1. Prove $\bar{L} \in$ NP.
2. Prove that $\forall L \in \text{NP} : \ L \leq_p \bar{L}$.

1. is usually easy (come up with a verification algorithm)
2. more difficult because of $\forall$! Infinite reductions!

Fortunately, $\leq_p$ is transitive!

If I know that a given language $\hat{L} \in$ NPC, it suffices to prove that

$$\underbrace{\hat{L}}_{\text{Known to be in NPC}} \leq_p \underbrace{\bar{L}}_{\text{Candidate to be in NPC}}.$$

If $\hat{L} \leq_p \bar{L}$, since $\forall L \in \text{NP} : \ L \leq_p \hat{L}$ (because $\hat{L} \in$ NPC) we have that for every language in NP, we can reduce it to $\hat{L}$ in polinomial time, and then reduce $\hat{L}$ to $\bar{L}$ in polynomial time. By transitivity of $\leq_p$, we have that for every language in NP, we can reduce it to $\bar{L}$ in polynomial time. So $\bar{L}$ is NP-hard, and if we can also show that $\bar{L} \in$ NP, then we have shown that $\bar{L} \in$ NPC.

$$\forall L \in \text{NP} : \ (L \leq_p \hat{L}) \wedge (\hat{L} \leq_p \bar{L}) \overset{\text{transitivity of } \leq_p}{\Longrightarrow} \forall L \in \text{NP} : \ L \leq_p \bar{L}.$$

A single reduction suffices!

Unfortunately, the transitivity trick cannot be used for the first language/problem to be placed in NPC! The first problem requires a "universal" reduction from a generic problem in NP, proving all languages in NP can be reduced to it. Computer scientist Cook was the first to show that the SAT problem is NP-complete, by showing that every language in NP can be reduced to SAT in polynomial time. This is known as Cook's Theorem.

RECAP

- Classes P and NP. P $\subseteq$ NP.
- Poly-time reducibility: $L_1 \leq_p L_2$. "relative difficulty" of problems:

$$\underbrace{(L_1 \leq_p L_2)} \wedge (L_2 \in \text{P}) \Rightarrow L_1 \in \text{P}.$$

- Classes NPH and NPC:

$$L' \in \text{NPC} : \quad (L' \in \text{NP}) \wedge \underbrace{(\forall L \in \text{NP} : \ L \leq_p L')}_{\text{NPH}}.$$

  Reducibility between languages suggests a "less difficult than" relation. NP-Hard languages are all languages such that all languages in NP reduce to them, NP-Complete languages are all NP-Hard lanugages that are also in NP.
- Facts:

$$(\text{P} \cap \text{NPC} \neq \varnothing) \Rightarrow \text{P} = \text{NP}.$$

$$L_1, L_2 \in \text{NPC} : \ (L_1 \leq_p L_2) \wedge (L_2 \leq_p L_1).$$

- Membership in NPC: use transitivity of $\leq_p$ by proving:

$$L_{\text{KNOWN-NPC}} \leq_p L_{\text{CANDIDATE}}.$$

- First language in NPC requires direct application of definition: ∞ reductions!

$$\forall L \in \text{NP}, \ L \leq_p L_{\text{KNOWN-NP}}.$$

  but after we put the forst language in NPC, we can prove that other problems are in NPC just by proving that they can be reduced to problems already in NPC.
- Remember that the direction of the reduction is important. We need to prove that the candidate NPC language is harder that a language that is already in NPC, not the other way around :

$$L_{\text{KNOWN-NPC}} \leq_p L_{\text{CANDIDATE}}.$$

15 Boolean Circuits

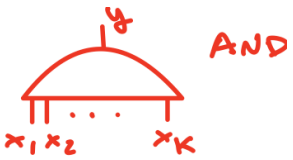
In 1971 Cook proved the first NP-completeness result on a problem in boolean logic. We'll work with an equivalent variant defined on boolean circuits.

We only look at combinatorial circuits, so acyclic interconnection of boolean gates, because a cycle in a circuit implies that the circuit has memory.

15.1 Boolean gates

A gate is an elementary switching circuit of three types:

1. AND



   output wire $y = 1 \iff$ input wires $x_1, x_2, \ldots, x_k$ all carry 1. Thus $y = \bigwedge_{i=1}^{k} x_i$.
2. OR



   output wire $y = 1 \iff$ at least one input wire carries 1. Thus $y = \bigvee_{i=1}^{k} x_i$.
3. NOT



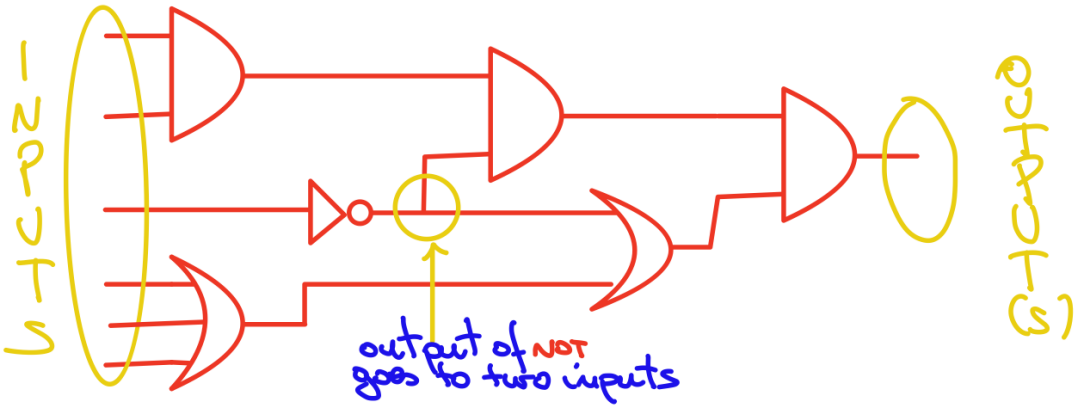
   output wire $y = 1 \iff$ input wire $x = 0$. Thus $y = \neg(x)$ $(= \bar{x})$.

Some subsets create a Elementary universal family of gates

15.2 Boolean Circuit

DEF: A Boolean Circuit (BC) is an acyclic interconnection of gates, where the output of a gate can become input to one or more other gates.

Acyclic because we are interested in combinatorial circuits



Wires that do not originate from the output of a gate are called inputs of the circuit. They are dangling wires. They are Level 0.

Wires that do not enter into a gate are called outputs of the circuit. We will mostly consider circuits with a single output.

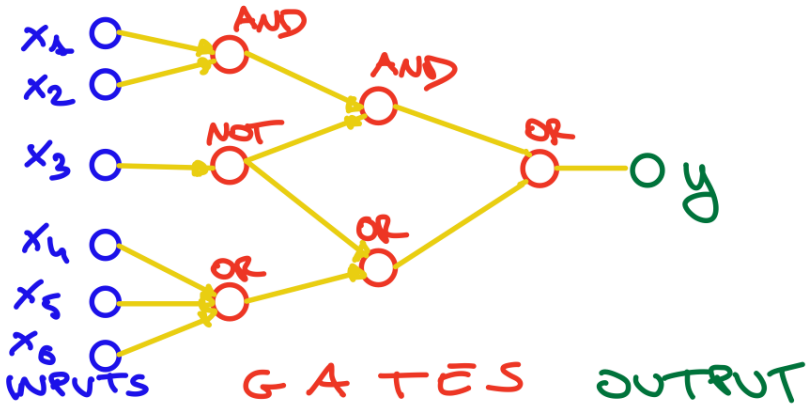
FAN-IN: max number of wires entering a gate over all gates in the circuit.

FAN-OUT: max number of gates fed by the same wire over all gates in the circuit.

A circuit can be represented by a directed graph (circuits are in a 1 to 1 correspondence on a directed graph).

$$C = (I \cup G \cup O, W).$$

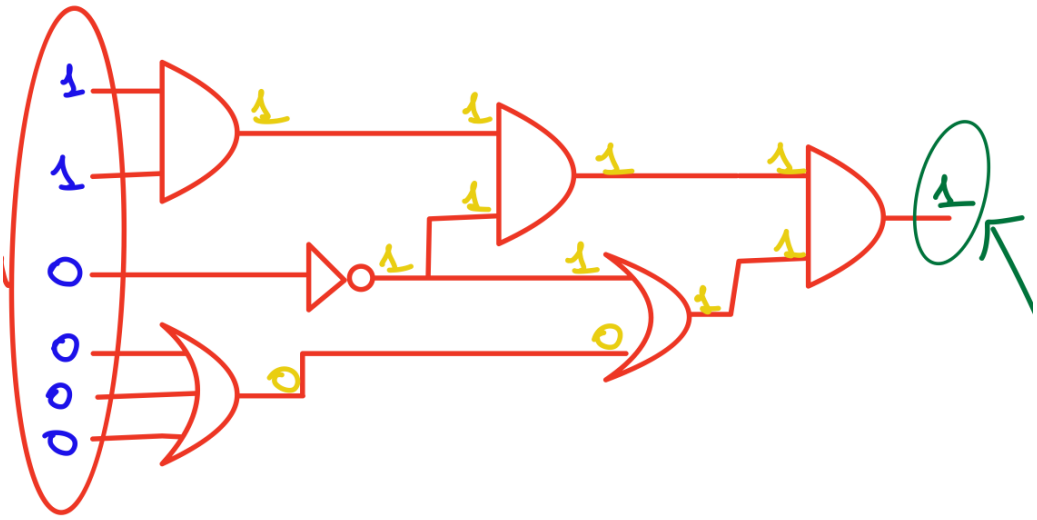
- The nodes are the inputs (I), the gated (G) and the outputs of the circuit (O). They are labelled according to what they represent.
- The set of edges contains one edge for each wire (W). The edges are directed.



15.3 Truth assignments

A truth-assignment is a boolean vector that associates to the inputs of a circuit a boolean value to each input.

Under a truth-assignment each wire in the circuit will switch to the correct boolean value according to the gates traversed. The circuit commutes.



The circuit implements a boolean function $y = C(x_1, x_2, \ldots, x_n)$. Y (the output) is the result of the values given in input to the function (the circuit)

For a circuit with n inputs, a truth assignment is a vector with one bit per input $\vec{b} \in \{0,1\}^n$.

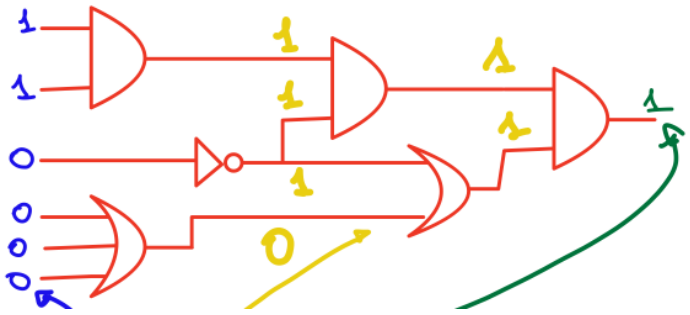
15.4 Configuration

After the circuit has commuted every wire in the circuit will carry a stable boolean value.

The set of boolean values carried by all wires of a circuit C under a truth-assignment $\vec{b}$ is called Configuration γ_C of C under $\vec{b}$.

Since the input is part of the configuration, and the truth assignemnt has N bits, there are 2 to the N possible truth assignments and therefore 2 to the N possible configurations.

DEF: A boolean circuit $C(x_1, x_2, \ldots, x_n)$ is satisfiable if there exists a truth assignment such that the output of the circuit has value 1 : $\vec{b} \in \{0,1\}^n : \ C(\vec{b}) = 1$.



C is satisfied by the truth assignment $\vec{b} = (1,1,0,0,0,0)$.

Configuration: $(1,1,0,0,0,0,1,1,1,0,1,1,1) = \gamma_C$ (in some prespecified order).

16 Boolean Circuit Satisfiability Problem BC-SAT

The boolean circuit satisfiability problem is a decision problem where the input is a boolean function associated to a circuit and the question is wether or not a truth assignment that satisfies

the circuit exists.

$$\text{I: } \langle C(x_1, \ldots, x_n) \rangle = \langle (I \cup G \cup O, W) \rangle, \quad \text{boolean circuit,}$$
$$\text{Q: } C \text{ is satisfiable? } \quad \exists \vec{b} \in \{0,1\}^n : \ C(\vec{b}) = 1?$$

BC-SAT is an important problem in practice (It is used to do circuit testing, circuit equivalence ...).

16.1 BC-SAT is a NP problem

THEOREM: BC-SAT $\in$ NP.

Proof steps:

- Make sure that the input is well formed (Is it the encoding of a combinatorial circuit?)
- Make sure that the candidate solution is a feasible configuration of the circuit : every wire that comes from the same gate has to have the same value and the input output relation of the gate must be correct according to the function of the gate.
- Return the value associated to the output in the certificate.

HOMEWORK : Correctness : The verifier returns the value of the certificate as output only if the certificate is valid, else returns 0. This is correct because if the certificate is valid and the output is 0 we do not have the guarantee that a valid certificate exists. Instead, if the value is 1, we just proved that certificate to be a valid certificate and therefore the circuit is satisfiable.

This is the proof we need to give to say that the language verified by this verifier is the set of all encodings of circuits that are satisfiable.

HOMEWORK : Running Time Prove that it is linear in the dimension of the input and the certificate. Going through the graph with an adjacency list of gates to check if the values are feasible is just a linear scan of the graph :

- Checking that all the outputs of a gate are the same is done by checking all outgoing edges of each node.
- Checking that the I/O relationship is valid can also be done with a check on all incoming edges and outgoing edges of a node.

This makes checks proportional on the size of the encoding (x+y)

VERIFY-BCSAT(x, y)

if $x \neq \langle (I \cup G \cup \{y_0\}, W) \rangle$ then return 0
then return 0

if $\exists w_i \neq w_j$ outgoing from same gate
  then return 0

for each gate $g \in G$
  if (I/O relation for g incorrect) then return 0

return wire value associated to y_0

$$T_{VB}(|x|, |y|) = \Theta(|x| + |y|) \quad \Rightarrow \quad \text{BC-SAT} \in \text{NP}.$$

17 Cook's Theorem

THEOREM (Cook, 1971)

$$\text{BC-SAT} \in \text{NPH}.$$

The proof of Cook's theorem is complex in the details but relies on a simple intuition.

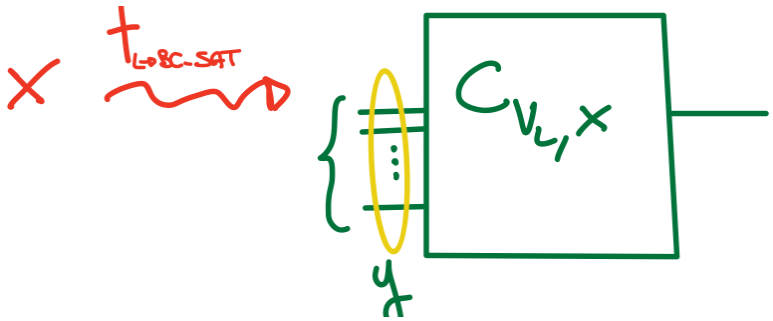
INFORMAL PROOF We need to show that given an arbitrary $L \in \text{NP}$, $L \leq_p L_{\text{BC-SAT}}$.

Given that we just know $L \in \text{NP}$, the only thing that we know is that, $V_L(x, y)$, a polynomial time verifier for L, exists. We also assume to know the bounds of the verifier, k1 and k2.

We must define a reduction function to reduce any instance of L to ann instance of BC SAT.

$$f_{L \to \text{BC-SAT}}(x) : \{0,1\}^* \longrightarrow \{0,1\}^*.$$

The reduction function builds a boolean circuit starting from V_L and x. ì



The shape of $C_{V_L,x}$ depends on V_L and x. Its inputs "represent" input y of V_L.

We will make sure that f is such that if x is in L, it will be mapped to a satisfiable circuit. Since we are creating the mapping function, we decide to only use short certificates.

We can upper bound the number of bits that we need in association to a specific circuit. The number of bits contributed by X is fixed. We only care about certificates Y that are smaller equal than size of x to the k1. If no positive certificates of that length exist, then there can't be a positive large certificate.

THE REASON WHY a short Y exists is that L is is a language in NP

17.1 Cook's Reduction

$$f(x) = \langle C_{V_L,x} \rangle \in L_{\text{BC-SAT}} \text{ (is satisfiable)} \iff \exists \text{ certificate } y \text{ of poly-length for } x$$
$$\iff x \in L.$$

IDEA: $C_{V_L,x}(y)$ "simulates" the computation performed by $V_L(x, y)$. **Problem :** We have to prove that this reduction is obtainable in polynomial time.

17.2 Buinding the circuit

A computer is basically a sequential circuit with memory. Can we emulate it with combinatorial circuits? Yes! We can emulate the random access aspect with a sequential circuit, and the memory aspect is dealt with by associating every single bit of memory to a wire.

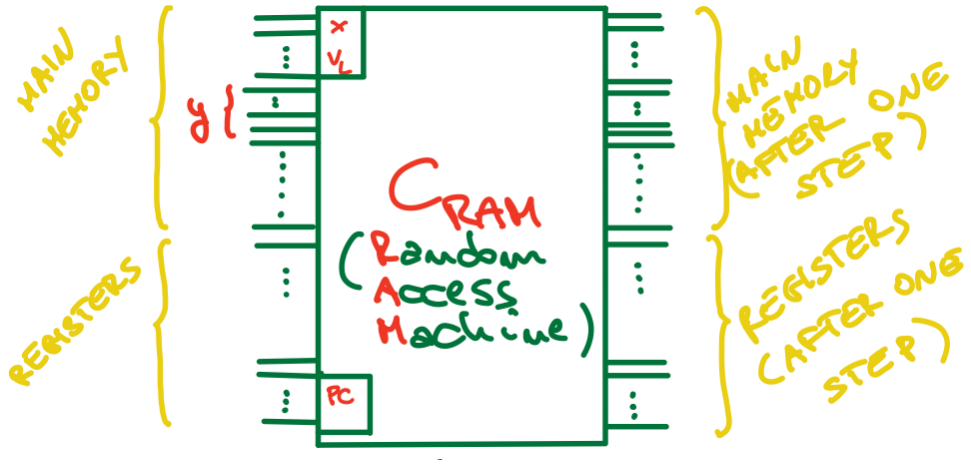
The initial value is passed on as input, and the corresponding output wire will hold the same bit's value after 1 step of computation of the random access machine.

The memory, register, and program counter initial values are hard coded into the first Circuit RAM of the circuit.

The whole circuit will have just one output : the return value of $V_L(x, y)$

We will not see how cook implemented the entire architecture parts like the ALU, the branch unit and the interpreter with only and or and not gates.

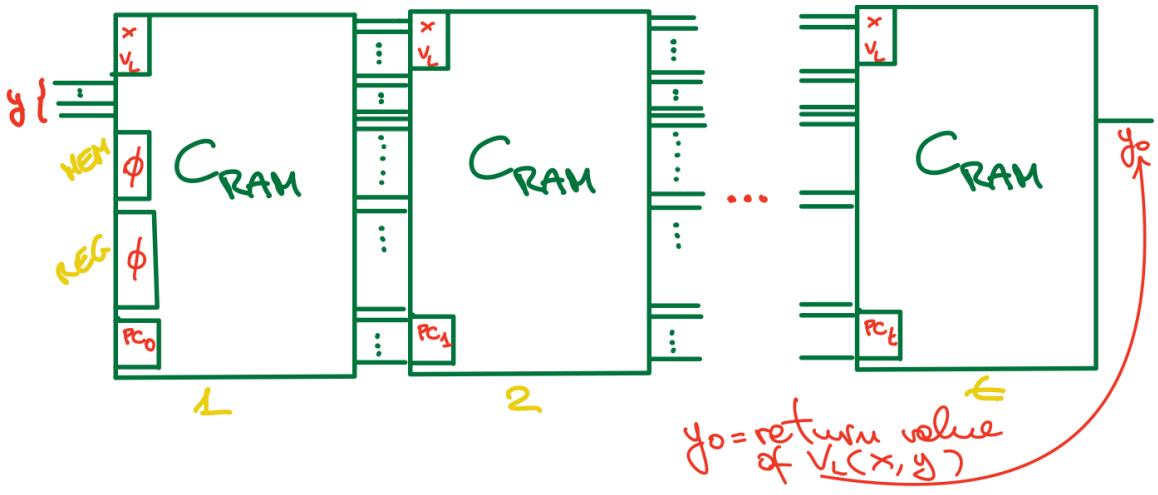
$C_{V_L,x}$ is built starting from a combinatorial circuit executing a single step of a Random Access Machine (RAM).



C_{RAM} implements the standard circuitry of a RAM (ALU, branches, etc. ...). Memory is "simulated" by passing along stored values on output wires.

We simulate an algorithm execution on a Random access machine with jusapposition of many C_RAM circuits : one circuit per computational step of the algorithm.

An algorithm executing t steps can be simulated by cascading t copies of C_{RAM}:



17.3 Crucial points:

1. $|\langle C_{\text{RAM}}\rangle|$ is polynomial in x and y.
   - circuitry polynomial in the memory size **NON CHIARO**.
2. It suffices to set $|y| = \Theta(|x|^{k_1})$.

3. (we can avoid examining every possible input length : we can limit from above the quantity of input wires.)
4. the number of wires is polynomial : parallelism with the implementation of the ALU. The Sum component which is the basic component to work needs a number of bits that is at most quadratic in the size of the operands.
5. It suffices to consider $t = \Theta((|x| + |y|)^{k_2})$.
6. (We will not need more than $(|x|+|y|)^{k_2}$ copies of the circuit to run the verification algorithm, because we know that since L is in NP it will run in polynomial time)

(k_1, k_2 from definition of poly-time verifier.)

We have a polynomial number of circuits with a polynomial number of wires : the product of a polynomial is a polynomial $\Rightarrow C_{V_L,x}$ has size polynomial in x! Therefore $f_{L\to\text{BC-SAT}}$ is PTC.

if x is accepted by a language in NP **if and only if** a certificate y exists such that y is small and the verification algorithm for L returns 1 for (x,y) **if and only if** our circuit built for VL on x returns 1 for the certificate y **if and only if** the circuit for VL on x is satisfiable (so belongs to BC SAT) **if and only if** the output of the reduction function from L to BC SAT given when x is the input belongs to BC SAT

Formally we have :

$$\begin{aligned} x \in L &\iff \exists \bar{y},\ |\bar{y}| = O(|x|^{k_1}) :\ V_L(x,\bar{y}) = 1 \\ &\iff C_{V_L,x}(\bar{y}) = 1 \\ &\qquad \{C_{V_L,x} \text{ simulates } V_L(x,y)\} \\ &\iff \langle C_{V_L,x}\rangle \in \text{BC-SAT} \\ &\iff f_{L\to\text{BC-SAT}}(x) \in \text{BC-SAT}. \end{aligned}$$

Thus $L \leq_p$ BC-SAT. This proves that BC-SAT $\in$ NPH.

We have come up with a uniform reduction of a general language in NP to a language associated to boolean circuit satisfiability.

In what follows we will prove that a number of problems are in NPC. The details of the reductions will teach us general approaches and highlight pitfalls to avoid.

18 Formula satisfiability problem (SAT)

A boolean formula $\phi(x_1, x_2, \ldots, x_n) : \{0,1\}^n \to \{0,1\}$ is a fully parenthesized expression built on

- n variables $x_1, x_2, \ldots, x_n$;
- binary operators: $\vee$ (OR), $\wedge$ (AND), $\to$ (IMPLIES), $\leftrightarrow$ (EQUALS);
- unary operator: $\neg$ (NOT).

The new operators implies and equals :

x, y	$x \to y$	$x \leftrightarrow y$
00	1	1
01	1	0
10	0	0
11	1	1

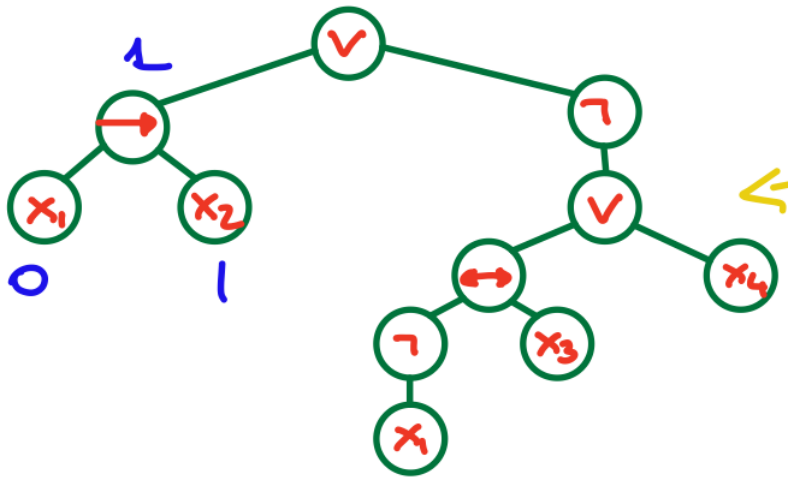
$$x \to y \ \equiv\ (\neg x) \vee y, \qquad x \leftrightarrow y \ \equiv\ (x \wedge y) \vee (\neg x \wedge \neg y).$$

18.1 formula encoding : a tree

We assume that the boolean formulas are always represented as their relative parenthesized trees.

$$\phi(x_1, x_2, x_3, x_4) = (x_1 \to x_2) \vee (\neg((\neg x_1 \leftrightarrow x_3) \vee x_4)).$$

Boolean formulas can be represented by the parenthesization tree. The leafs are the variables; the internal nodes are operators.



18.2 Boolean formula truth assignment

A truth-assignment is a vector of boolean values $\vec{b} \in \{0,1\}^n$, one for each variable. The value $\phi(\vec{b})$ can be computed in time linear in the formula's size by using the tree. You go up on the tree and progressively associate every internal node with the value obtained from the truth table of the operator of that node according to the values of its children.

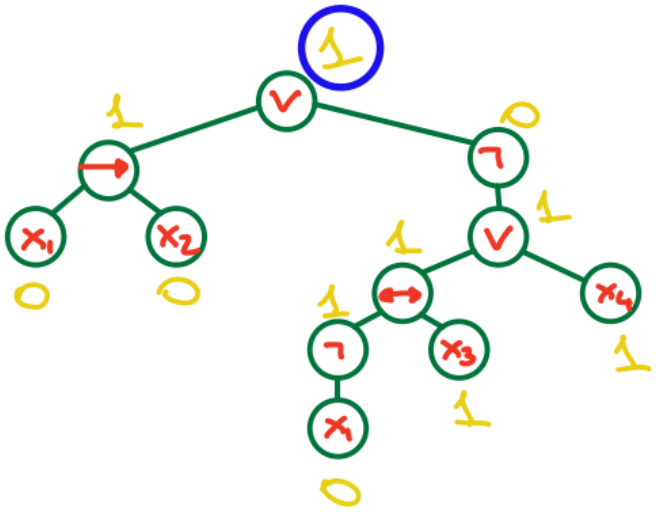
A satisfying assignment $\vec{b}$ is such that the formula under that truth assignment is true : $\phi(\vec{b}) = 1$. If $\vec{b}$ exists, ϕ is said satisfiable.

18.3 Formula SAT problem formulation

I: $\langle \phi(x_1, \ldots, x_n) \rangle$, $\quad \phi$encoding of the boolean formula,
Q: is ϕ satisfiable?

EXAMPLE: $\vec{b} = (0, 0, 1, 1)$.

$$\phi(x_1, x_2, x_3, x_4) = (x_1 \to x_2) \vee (\neg((\neg x_1 \leftrightarrow x_3) \vee x_4)).$$



ϕ is satisfiable $[\phi(\vec{b}) = 1]$.

Very similar to BC-SAT (in fact, this is Cook's original problem). Nonetheless, the reduction is nontrivial.

1. SAT $\in$ NP: Trivial: for $\langle \phi(x_1, \ldots, x_n) \rangle$ a candidate certificate is $\vec{b} \in \{0,1\}^n$. $\phi(\vec{b})$ can be computed in linear time using the tree representation.

- Check wether the first input represents a tree
- check weteher the second input is a truth assignment
- Implement bottom up ascent to check if the truth assignment is a positive one.

2. SAT $\in$ NPH. We show that BC-SAT $\leq_p$ SAT.

Reduction function f: must transform a circuit C into a formula ϕ_C:

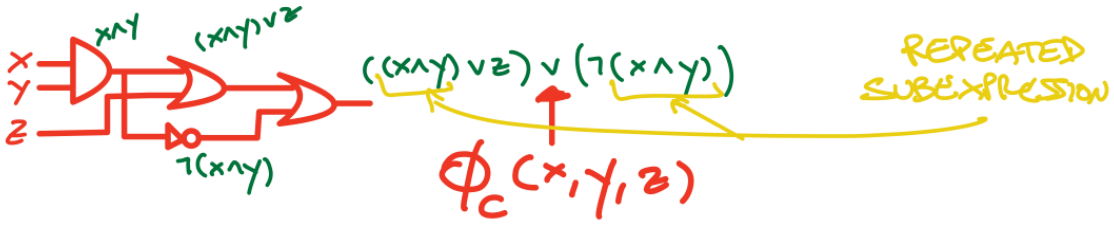
$$C \text{ is satisfiable} \iff f(C) = \phi_C \text{ is satisfiable},$$

$$\langle C \rangle \in \text{BC-SAT} \iff \langle \phi_C \rangle = f(\langle C \rangle) \in \text{SAT}.$$

18.4 Wrong Exponential Reduction

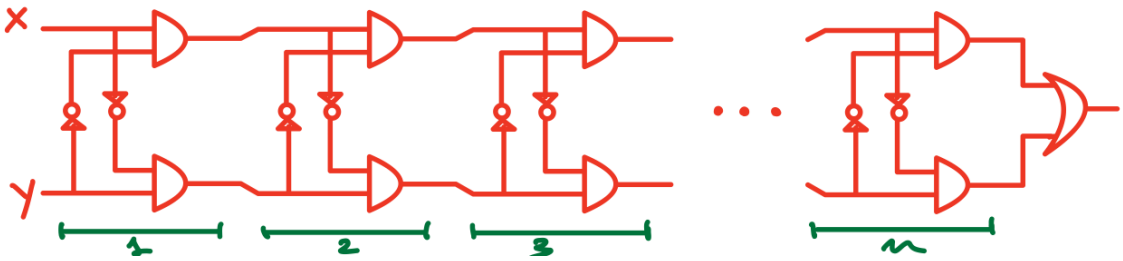
Seems easy: straightforward method: compute the subexpression associated to each wire. $\phi_C =$ expression associated to output wire.

EXAMPLE:



PROBLEM: Repeated subexpressions may blow-up the length of the formula exponentially!

CONSIDER:



Two-input circuit made of n stages. $|C(x_1, x_2)| = \Theta(n)$.

Outputs of Stage 1: $(x \wedge (\neg y)), \quad ((\neg x) \wedge y)$.

Outputs of Stage 2:

$$(x \wedge (\neg y)) \wedge (\neg((\neg x) \wedge y)), \qquad (\neg(x \wedge (\neg y))) \wedge ((\neg x) \wedge y),$$

etc.

The number of symbols in the subexpressions of Stage i is more than double than that of Stage $i-1$. This is because we have repeated subexpressions : Both outputs of the gates go into both inputs of the next gates, so if we have 2 outputs we have to repeat them twice because they will appear in both inputs of the next gates.

$$\Rightarrow \begin{cases} \text{Size}(i) \geq 2\,\text{Size}(i-1), \\ \text{Size}(1) = c. \end{cases}$$

$$\Rightarrow |\phi_C(x, y)| \geq \text{Size}(n) \geq c \cdot 2^{n-1}!$$

Solved the recurrence by unfolding : $|C| = \Theta(n)$ but $|\phi_C(x, y)| = \Omega(2^n)$.

This reduction cannot be ptc! (I can't generate an exponential number of symbols in polynomial time!)

We need a more sophisticated approach!

$$C = (\underbrace{I}_{x_i} \cup \underbrace{G}_{y_g\ \forall g \in G} \cup \underbrace{\{y_0\}}_{y_0}, W).$$

New variables y_g : $g \in G$ (associated to G).

RECAP

- COOK'S THEOREM: BCSAT $\in$ NPH.
  – "simulate" $V_L(x, y)$ with $C_{V_L,x}(y)$.
  A certificate y_x for $x \in L$ becomes a satisfying assignment for $f(x) : \langle C_{V_L,x}(y) \rangle$.
- FORMULA SATISFIABILITY (SAT): BC-SAT $\leq_p$ SAT.
  – Trivial reduction based on subexpressions not ptc (poly-time computable).

We need a more sophisticated approach!

$$C = (\underbrace{I}_{x_i} \cup \underbrace{G}_{y_g\ \forall g \in G} \cup \underbrace{\{o\}}_{y_0}, W).$$

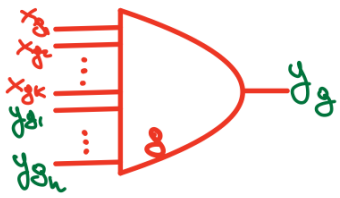
New variables y_g : $g \in G$.

We add extra variables y_g associated to the gates, describing the output of each gate $g \in G$ and create a formula that reflects valid configurations of the circuit.

RECALL: A configuration is the set of all bits on the wires under a valid switching of the circuit: ($\underbrace{\vec{b}_1}_{\vec{x}}$, $\underbrace{\vec{b}_2}_{\vec{y}\text{ (gate outputs)}}$).

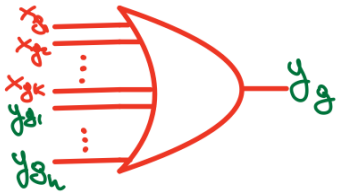
For each gate $g \in G$ we introduce a sub-formula Γ_g describing the input-output relation of the gate:

- AND gate:



$$\Gamma_g : \; y_g \underbrace{\leftrightarrow}_{\text{“equation”}} x_{g_1} \wedge \cdots \wedge x_{g_k} \wedge y_{g_1} \wedge \cdots \wedge y_{g_h}.$$

- OR gate:



$$\Gamma_g : \; y_g \leftrightarrow x_{g_1} \vee \cdots \vee x_{g_k} \vee y_{g_1} \vee \cdots \vee y_{g_h}.$$

- NOT gate:



$$\Gamma_g : \; y_g \leftrightarrow \neg(z_{g_1}).$$

Observe that subexpression Γ_g is true under a truth assignment iff y_g is assigned the correct output value w.r.t. the values assigned to the gate's inputs.

We can then obtain the formula

$$\psi(\vec{x}, \vec{y}) = \bigwedge_{g \in G} \Gamma_g.$$

Formula ψ describes a configuration of the circuit C:

$$\begin{aligned} \psi(\vec{b}_1, \vec{b}_2) = 1 &\iff \text{the values of the outputs of the gates of } C(\vec{b}_1) \text{ are } (b_2)_g : g \in G \\ &\iff (\vec{b}_1, \vec{b}_2) \text{ is a valid configuration.} \end{aligned}$$

OBSERVATION: $\psi(\vec{x}, \vec{y})$ is always satisfiable (HOMEWORK: how many satisfying assignments?)

We have that

$$|\langle \psi(\vec{x}, \vec{y}) \rangle| = \Theta(|\langle C(\vec{x}) \rangle|)$$

since each subexpression uses a number of symbols linear in the size of the corresponding gate!

The reduction function f uses ψ as follows:

$$f(\langle C(\vec{x})\rangle) = \phi_C(\vec{x},\vec{y}) = \psi(\vec{x},\vec{y}) \wedge \underbrace{y_0}_{\text{variable associated to gate of output wire}}.$$

$$|\langle \phi_C(x,y)\rangle| = \Theta(|\langle \psi(\vec{x},\vec{y})\rangle|) = \Theta(|\langle C(x)\rangle|).$$

Thus f is ptc.

We are left to prove that $x \in L_1 \iff f(x) \in L_2$.

$$\begin{cases} x \in L_1 \Rightarrow f(x) \in L_2, \\ f(x) \in L_2 \Rightarrow x \in L_1 \end{cases} \quad \text{OR} \quad \begin{cases} x \in L_1 \Rightarrow f(x) \in L_2, \\ x \notin L_1 \Rightarrow f(x) \notin L_2. \end{cases}$$

$$\langle C(\vec{x})\rangle \in \text{BC-SAT} \iff f(\langle C(\vec{x})\rangle) = \langle \phi_C(\vec{x},\vec{y})\rangle \in \text{SAT}.$$

($\Rightarrow$) If $\langle C(\vec{x})\rangle \in$ BC-SAT then $\exists \vec{b_1} \in \{0,1\}^n : C(\vec{b_1}) = 1$. Consider the configuration of the circuit under $\vec{b_1}$, and let $\vec{b_2}$ be the values carried by the wires at the outputs of the gates. We have $\psi(\vec{b_1},\vec{b_2}) = 1$. Moreover, since $C(\vec{b_1}) = 1$ we have $\underbrace{b_0}_{y_0} = 1$.

$$\Rightarrow \phi_C(\vec{b_1},\vec{b_2}) = \psi(\vec{b_1},\vec{b_2}) \wedge b_0 = 1 \Rightarrow f(\langle C(\vec{x})\rangle) \in \text{SAT}.$$

($\Leftarrow$) $f(\langle C(\vec{x})\rangle) \in \text{SAT} \Rightarrow \exists \vec{b_1},\vec{b_2} : \phi_C(\vec{b_1},\vec{b_2}) = 1$.

1. $\psi(\vec{b_1},\vec{b_2}) = 1$: the truth assignments represent a legal configuration $(\vec{b_1},\vec{b_2})$ of the circuit C.
2. Under $(\vec{b_1},\vec{b_2}) : b_0 = 1 \Rightarrow C(\vec{b_1}) = 1$.

Thus $\langle C(\vec{x})\rangle \in$ BC-SAT.

LESSON LEARNED: We have to make sure that the reduction is poly-time computable!

SIMILAR PITFALL: $f_{L' \to L}$ cannot possibly know whether $x \in L'$ or $x \notin L'$ unless $L' \in$ P! This is never the case if you are trying to prove that $L \in$ NPC (IMPORTANT).

Next reduction is a special case of a large family: reductions by restriction.

These reductions are used to prove that an NPC problem stays NPC even if we restrict the set of admissible instances.

DEF Given a set of boolean variables $\{x_1, x_2, \ldots, x_n\}$:

- A literal is either a variable x_i or its negation $\neg x_j$ ($\bar{x}_j$). We will use y to denote a literal.
- A clause is a disjunction of 3 distinct literals, e.g. $C_i = x_1 \vee \bar{x}_3 \vee x_5$.
- A 3-CNF formula $\phi(x_1,\ldots,x_n)$ is a conjunction of clauses built on the n variables.

E.g. $\phi(x_1,x_2,x_3,x_4) = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3)$.

3-CNF-SAT:

I: $\langle \phi(x_1,x_2,\ldots,x_n) = \bigwedge_{i=1}^{m} C_i \rangle$, ϕ 3-CNF formula.

Q: Is $\phi(\vec{x})$ satisfiable?

We will prove that 3-CNF-SAT $\in$ NPC.

Meaning: Even restricting the shape of the formula to a very simple one, the problem stays difficult!

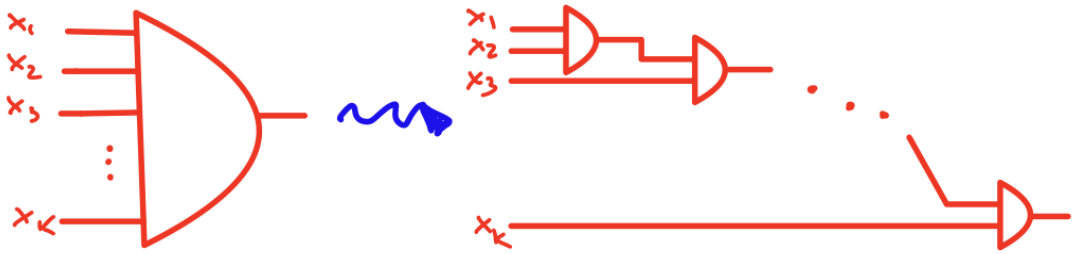
3-CNF-SAT $\in$ NP (trivial, exercise).

3-CNF-SAT $\in$ NPH:

We reduce again from BC-SAT, using a similar idea to BC-SAT $\leq_p$ SAT.

- PREPROCESSING STEP: we modify C to get an equivalent one:

Replace each AND or OR gate with fan-in ≥ 3 with a cascade of gates with fan-in 2, e.g.,



The resulting circuit C' is equivalent to C. Moreover $|\langle C' \rangle| = \Theta(|\langle C \rangle|)$ (# new gates $\leq$ # wires of C).

- Next, we write the formula $\psi(\vec{x}, \vec{y}) \wedge y_0$ as done for formula satisfiability. We have four types of subexpressions Γ:

$$y_i \leftrightarrow w_i^1 \vee w_i^2, \qquad y_i \leftrightarrow w_i^1 \wedge w_i^2, \qquad y_i \leftrightarrow \bar{w}_i, \qquad y_0.$$

  (w_i either x_k or y_j.)
- We can re-write each subexpression as a conjunction of disjunctions:

$$\begin{aligned} y_i \leftrightarrow w_i^1 \vee w_i^2 &\equiv (y_i \vee \bar{w}_i^1) \wedge (y_i \vee \bar{w}_i^2) \wedge (\bar{y}_i \vee w_i^1 \vee w_i^2), \\ y_i \leftrightarrow w_i^1 \wedge w_i^2 &\equiv (\bar{y}_i \vee w_i^1) \wedge (\bar{y}_i \vee w_i^2) \wedge (y_i \vee \bar{w}_i^1 \vee \bar{w}_i^2), \\ y_i \leftrightarrow \bar{w}_i &\equiv (y_i \vee w_i) \wedge (\bar{y}_i \vee \bar{w}_i). \end{aligned}$$

Equivalence verified via truth tables (HOMEWORK).

E.g.,

y_i	w_i	$y_i \leftrightarrow \bar{w}_i$	$A : y_i \vee w_i$	$B : \bar{y}_i \vee \bar{w}_i$	$A \wedge B$
0	0	0	0	1	0
0	1	1	1	1	1
1	0	1	1	1	1
1	1	0	1	0	0

$y_i \leftrightarrow \bar{w}_i = A \wedge B$.

- After this transformation, $\psi(x; y) \wedge y_0$ has been rewritten as a conjunction of disjunctive clauses. However, some of the clauses have < 3 literals!
- We add "dummy" variables to bring all clauses to 3 literals:

$$\begin{aligned} (y \vee w) &= (y \vee w) \vee 0 = (y \vee w) \vee (z \wedge \bar{z}) \\ &= (y \vee w \vee z) \wedge (y \vee w \vee \bar{z}). \end{aligned}$$

(For clause y_0 repeat twice.)

- We have obtained a 3-CNF-FORMULA $\phi_{C'}(\vec{x}, \vec{y}, \vec{z})$ equivalent to $\psi_{C'}(\vec{x}, \vec{y}) \wedge y_0$.

$$\begin{aligned} C \in \text{BC-SAT} &\iff C' \in \text{BC-SAT} \\ &\iff \psi_{C'}(\vec{x}, \vec{y}) \wedge y_0 \in \text{SAT} \\ &\iff \phi_{C'}(\vec{x}, \vec{y}, \vec{z}) \in \text{3-CNF-SAT}. \end{aligned}$$

LESSON LEARNED: Reductions by restriction must "modify" f so that it maps into the set of restricted instances.

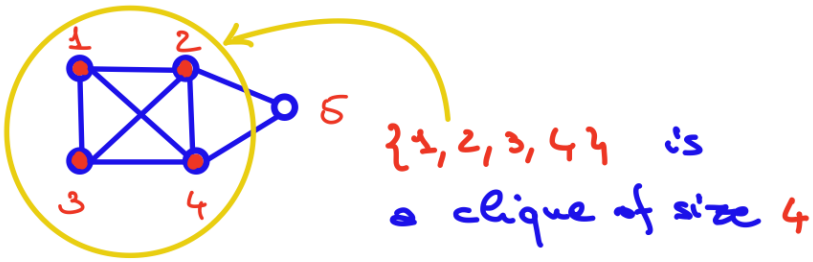
PROBLEMS ON GRAPHS

We will prove the NP-Completeness of three decision problems on graphs, related to very important optimization problems.

DEF Given $G = (V, E)$ undirected, a clique of size k is a subset $V' \subseteq V$ of k nodes such that each pair of distinct nodes in V' is connected by an edge (clique: complete subgraph).

$$V' \subseteq V : \ (|V'| = k) \wedge (\forall u \neq v \in V : (u \in V') \wedge (v \in V') \Rightarrow \{u, v\} \in E).$$

EXAMPLE:



We have:

CLIQUE

I: $\langle G = (V, E), k \rangle$, G undirected graph, $1 \leq k \leq |V|$.

Q: Does G contain a clique of size k?

The optimization version of this problem requires determining the clique of max size.

QUESTION: Why doesn't the decision problem ask "size $\geq k$"? [Argue that the two questions are equivalent.]

Important application in bioinformatics, computational chemistry, social network analysis.

Let's prove that CLIQUE $\in$ NPC.

1. CLIQUE $\in$ NP trivial. Candidate certificate: set of k nodes. [Write $V_{\text{CLIQUE}}(x, y)$ as an exercise.]
2. CLIQUE $\in$ NPH. Reduction from 3-CNF-SAT.

$$\underbrace{\text{3-CNF-SAT}}_{\text{Known NPH}} \leq_p \underbrace{\text{CLIQUE}}_{\text{candidate}}.$$

We start from $\phi(x_1, x_2, \ldots, x_m) = C_1 \wedge C_2 \wedge \cdots \wedge C_m$ with $C_i = y_1^i \vee y_2^i \vee y_3^i$, $y_j^i \in \{x_k, \bar{x}_k : 1 \leq k \leq n\}$.

We have to define a reduction function that takes ϕ and returns an instance of CLIQUE in poly-time:

$$f(\langle \phi(x_1, \ldots, x_n) \rangle) = \langle G_\phi, k_\phi \rangle.$$

1. We set $k_\phi = m$ (# of clauses of ϕ).
2. $G_\phi = (V_\phi, E_\phi)$.

V_ϕ: (One node for each literal y_j^i, $1 \leq i \leq m$, $1 \leq j \leq 3$):

$$V_\phi = \{v_j^i : 1 \leq i \leq m, 1 \leq j \leq 3\} \Rightarrow |V_\phi| = 3m.$$

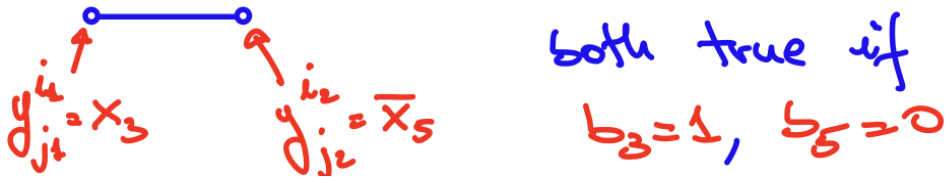
E_ϕ:

$$\{v_{j_1}^{i_1}, v_{j_2}^{i_2}\} \in E_\phi \iff (i_1 \neq i_2) \wedge (y_{j_1}^{i_1} \neq \neg y_{j_2}^{i_2}).$$

There is an edge between two nodes if their corresponding literals belong to distinct clauses and are not the negation of each other (e.g. $y_{j_1}^{i_1} = x_1$, $y_{j_2}^{i_2} = \bar{x}_1$).

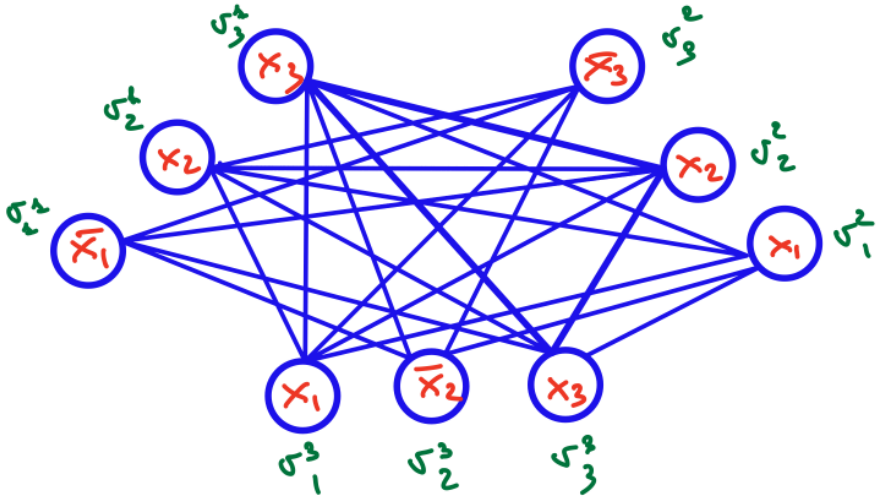
INTUITION: Two literals corresponding to an edge can be made both true under the same truth assignment.

E.g.,



EXAMPLE:

$$\phi(x_1, x_2, x_3) = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \bar{x}_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3).$$



Running time: $\Theta(m^2) = \Theta(|\langle\phi\rangle|^2)$.

Each node is compared to all other nodes to determine the edges.

$\Rightarrow f$ is ptc.

RECAP

- BC-SAT $\in$ NPC (Cook, 1971).
- SAT $\in$ NPC (BC-SAT $\leq_p$ SAT, make sure that $f(x)$ is polynomial).
- 3-CNF-SAT $\in$ NPC (reduction by restriction).
- CLIQUE $\in$ NPC (3-CNF-SAT $\leq_p$ CLIQUE, "context switch"):

$$f(\langle\phi(x_1, \ldots, x_n) = \bigwedge_{i=1}^{m} C_i\rangle) = \langle G_\phi, k_\phi\rangle, \qquad C_i = y_1^i \vee y_2^i \vee y_3^i.$$

1. $k_\phi = m$.

2. $G_\phi = (V_\phi, E_\phi)$.

$$V_\phi = \{v_j^i : 1 \leq i \leq m, 1 \leq j \leq 3\} \Rightarrow |V_\phi| = 3m.$$

One node per literal occurrence.

E_ϕ:

$$\{v_{j_1}^{i_1}, v_{j_2}^{i_2}\} \in E_\phi \iff (i_1 \neq i_2) \wedge (y_{j_1}^{i_1} \neq \neg y_{j_2}^{i_2}).$$

Edges between literals of different clauses that can both be True under the same truth assignment.

We have to show that:

$$\langle\phi\rangle \in \text{3-CNF-SAT} \iff f(\langle\phi\rangle) = \langle G_\phi, k_\phi\rangle \in \text{CLIQUE}.$$

($\Rightarrow$) $\langle\phi\rangle \in$ 3-CNF-SAT: there exists a satisfying assignment $\vec{b} \in \{0,1\}^n : \phi(\vec{b}) = 1$. Since ϕ is in 3-CNF, all clauses must be True under $\vec{b} \Rightarrow$ under $\vec{b}$ there is a true literal in each clause. Let these literals be:

$$\{y_{j_1}^1, y_{j_2}^2, \ldots, y_{j_m}^m\}, \qquad 1 \leq j_i \leq 3, \quad 1 \leq i \leq m,$$

and let $V' = \{v_{j_1}^1, v_{j_2}^2, \ldots, v_{j_m}^m\}$ (corresponding nodes in G_ϕ).

For two distinct nodes $v_{j_{i_1}}^{i_1}, v_{j_{i_2}}^{i_2} \in V'$, we have:

$$(i_1 \neq i_2) \wedge (y_{j_{i_1}}^{i_1} \text{ and } y_{j_{i_2}}^{i_2} \text{ both True under } \vec{b}) \Rightarrow \{v_{j_{i_1}}^{i_1}, v_{j_{i_2}}^{i_2}\} \in E.$$

Thus V' is a CLIQUE of size $k_\phi = m$.

$$\Rightarrow f(\langle\phi\rangle) = \langle G_\phi, k_\phi\rangle \in \text{CLIQUE}.$$

($\Leftarrow$) $f(\langle\phi\rangle) = \langle G_\phi, k_\phi\rangle \in$ CLIQUE.

Let us build a satisfying assignment $\vec{b}$ starting from a clique $V' = \{v_{j_1}^1, v_{j_2}^2, \ldots, v_{j_m}^m\}$.

For $1 \leq i \leq m$:

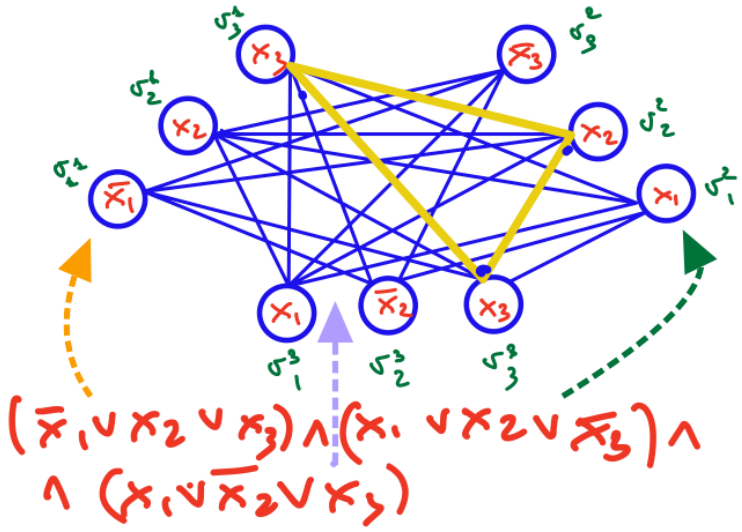
1. If $y_{j_i}^i = x_k$ then set $b_k = 1$.
2. If $y_{j_i}^i = \bar{x}_k$ then set $b_k = 0$.

If some value b_ℓ is still unfixed, set it to an arbitrary value.

Assignment $\vec{b}$ is well-defined since I cannot set $b_k = 0$ for some i_1 and $b_k = 1$ for some i_2, or otherwise $y_{j_{i_1}}^{i_1} = \bar{y}_{j_{i_2}}^{i_2}$ (impossible, since V' is a clique).

Under $\vec{b}$ there is at least one true literal in each clause

$$\Rightarrow \phi(\vec{b}) = 1 \Rightarrow \langle\phi\rangle \in \text{3-CNF-SAT}.$$



A 3-clique contains 3 literals (one per clause) that can be made all True under the same truth assignment.

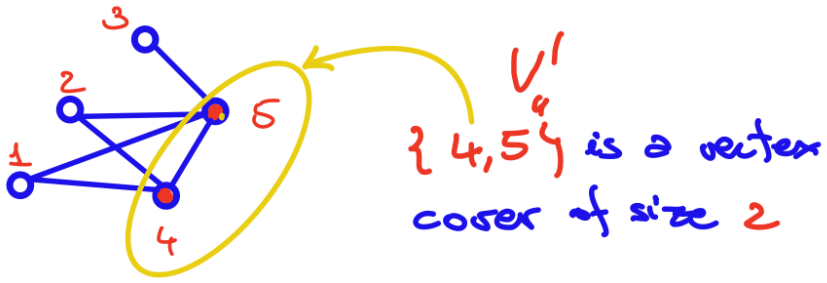
$$v_3^1 \leftrightarrow x_3 \longrightarrow b_3 = 1, \qquad v_2^2 \leftrightarrow x_2 \longrightarrow b_2 = 1, \qquad v_3^3 \leftrightarrow x_3 \longrightarrow b_3 = 1, \qquad b_1 = 0/1.$$

We introduce two extra problems on graphs:

DEF: Given $G = (V, E)$ undirected, a vertex cover of size k is a subset of nodes $V' \subseteq V$, $|V'| = k$ such that each edge in E has at least one endpoint in V'.

$$V' \subseteq V,\ |V'| = k:\ \forall u \neq v \in V:$$
$$\{u, v\} \in E \Rightarrow (u \in V') \vee (v \in V').$$

EXAMPLE:



We have:

VERTEX COVER (VC)

I: $\langle G = (V, E), k \rangle$, G undirected graph, $1 \leq k \leq |V|$.

Q: Does G contain a vertex cover of size k?

The optimization version of this problem requires determining the vertex cover of min size.

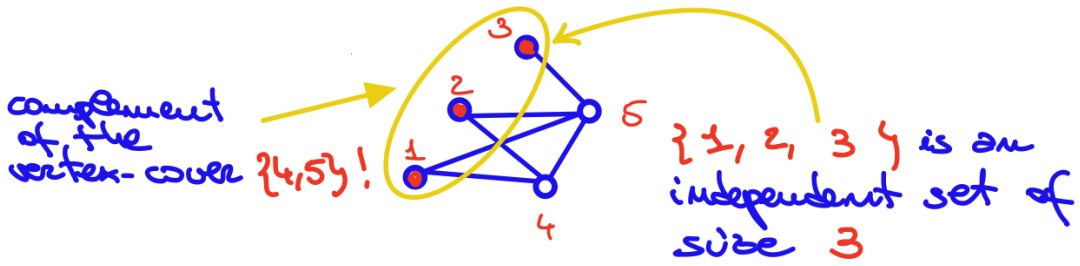
A vertex cover identifies a (possibly small) set of nodes from where all edges can be controlled. Crucial in networking, distributed computing, etc.. . .

Second problem:

DEF: Given $G = (V, E)$ undirected, an independent set of size k is a subset of nodes $V' \subseteq V$, $|V'| = k$ such that there is no edge in E with both endpoints in V'!

$$V' \subseteq V,\ |V'| = k:\ \forall u \neq v \in V:$$
$$(u \in V') \wedge (v \in V') \Rightarrow \{u, v\} \notin E.$$

EXAMPLE:



We have:

INDEPENDENT SET (IS)

I: $\langle G = (V, E), k \rangle$, G undirected graph, $1 \leq k \leq |V|$.

Q: Does G contain an independent set of size k?

The optimization version of this problem requires determining the independent set of max size.

An independent set is somewhat the "negative" of a clique, with no edges connecting any two of its nodes. Important primitive for graph-based computations.

The three problems share many similarities. We will see that reductions will be mostly "syntactic", based on logical manipulation of the definitions of clique, vertex-cover, independent-set.

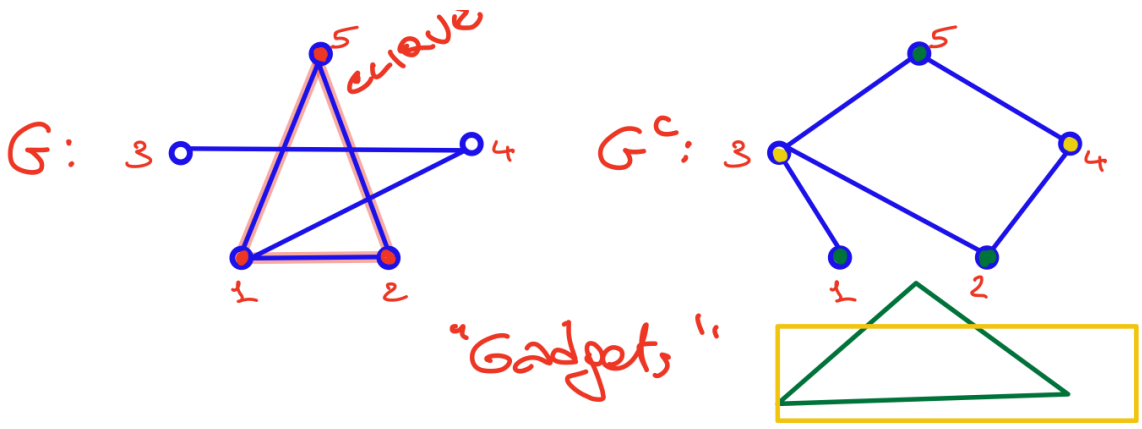
We will make use of the complement of a graph.

DEF: Given $G = (V, E)$ its complement $G^c = (V, E^c)$ is the graph with the same node-set and

$$E^c:\ \{\{u, v\} : u \neq v \in V,\ \{u, v\} \notin E\}.$$

G^c is the "negative" of G w.r.t. edges.

EXAMPLE



OBSERVE: Clique $\{1, 2, 5\}$ in G implies that:

1. $V - \{1,2,5\} = \{3,4\}$ is a vertex cover in G^c (all edges between $\{1,2,5\}$ disappear in E^c, thus each edge in E^c has an endpoint in $\{3,4\}$).
2. $\{1,2,5\}$ is an independent set in G^c (there cannot be edges between $\{1,2,5\}$ in E^c).

These observations suggest the following two reductions!

1. CLIQUE $\leq_p$ VC using:

$$f_{C\to \mathrm{VC}}(\langle G=(V,E),k\rangle) = \langle G^c=(V,E^c), |V|-k\rangle.$$

2. CLIQUE $\leq_p$ IS using:

$$f_{C\to \mathrm{IS}}(\langle G=(V,E),k\rangle) = \langle G^c=(V,E^c), k\rangle.$$

Clearly, $f_{C\to \mathrm{VC}}$ and $f_{C\to \mathrm{IS}}$ are ptc (the construction of $\langle G^c, |V|-k\rangle$ and $\langle G^c, k\rangle$ requires time $O(|\langle G,k\rangle|^2)$).

Let's prove that $f_{C\to \mathrm{VC}}$ is a reduction by logical manipulations of the definition of CLIQUE:

$$x = \langle G=(V,E),k\rangle \in \mathrm{CLIQUE}$$

$$\iff \exists V' \subseteq V,\ (|V'|=k) \wedge \big(\forall u \neq v \in V : (u\in V') \wedge (v \in V') \Rightarrow \{u,v\} \in E\big).$$

{transform using contrapositive} + $\{\neg(a\wedge b) = \neg a \vee \neg b\}$

$$\iff \exists V' \subseteq V,\ (|V'|=k) \wedge \big(\forall u \neq v \in V' : \{u,v\} \notin E \Rightarrow (u\notin V') \vee (v \notin V')\big).$$

$$\{u,v\} \notin E \iff \{u,v\} \in E^c, \qquad u \notin V' \iff u \in V - V'.$$

$$\iff \exists V' \subseteq V,\ (|V'|=k) \wedge \big(\forall u \neq v \in V' : \{u,v\} \in E^c \Rightarrow (u\in V-V') \vee (v \in V-V')\big).$$

{now, set $V'' = V - V' : |V''| = |V| - k$}

$$\iff \exists V'' \subseteq V,\ (|V''|=|V|-k) \wedge \big(\forall u \neq v \in V' : \{u,v\} \in E^c \Rightarrow (u\in V'') \vee (v \in V'')\big).$$

{this says that V'' is a VC of G^c}

$$\iff f(x) = \langle G^c=(V,E^c), |V|-k\rangle \in \mathrm{VC}.$$

REMARK: The proof proceeds by logical equivalences (can be read both $\Downarrow$ and $\Uparrow$).

Let's use the same technique to prove that $f_{C\to \mathrm{IS}}$ is a reduction:

$$x = \langle G=(V,E),k\rangle \in \mathrm{CLIQUE}$$

$$\iff \exists V' \subseteq V,\ (|V'|=k) \wedge \big(\forall u \neq v \in V' : (u\in V') \wedge (v \in V') \Rightarrow \{u,v\} \in E\big)$$

$$\iff \exists V' \subseteq V,\ (|V'|=k) \wedge \big(\forall u \neq v \in V' : (u\in V') \wedge (v \in V') \Rightarrow \{u,v\} \notin E^c\big).$$

{this says that V' is an IS of G^c}

$$\iff f(x) = \langle G^c=(V,E^c), k\rangle \in \mathrm{IS}.$$

EXERCISE Using the same technique, prove that VC $\leq_p$ IS.

These reductions suggest that CLIQUE, VC and IS are mostly equivalent problems BUT this is true only for what concerns exact solutions! We will argue that the optimization versions of the three problems are radically different w.r.t. approximation. In particular:

MINIMUM VC is "easy" to approximate.

MAXIMUM CLIQUE/IS are "hard" to approximate.

We now introduce a problem in the realm of COMPUTER ARITHMETIC:

SUBSET-SUM (SS)

I: $\langle S, t\rangle : S \subseteq \mathbb{N}$ finite, $t \in \mathbb{N}$ ("target").

Q: $\exists S' \subseteq S : \sum_{s\in S'} s = t$?

The problem asks whether there is a subset of the input set whose sum is equal to t (the target).

EXAMPLE:

$$\langle\{10, 15, 4, 3, 17\}, 24\rangle \in L_{\text{SS}}$$

since $24 = 4 + 3 + 17$ ($S' = \{3, 4, 17\}$).

REMARK: $|\langle S, t\rangle|$ is the total number of bits needed to encode $s \in S$ $\forall s$ and t:

$$|\langle S, t\rangle| = \Theta\Big(\sum_{s\in S} \log s + \log t\Big).$$

Clearly, SS $\in$ NP.

Candidate certificate: $y = \langle S'\rangle$, $S' \subseteq S$.

Let us prove that SS $\in$ NPH by proving that 3-CNF-SAT $\leq_p$ SS.

The reduction is much more complex than the other ones seen before.

Given a 3-CNF formula:

$$\phi(x_1, x_2, \ldots, x_n) = C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

with $C_i = y_1^i \vee y_2^i \vee y_3^i$, $y_j^i \in \{x_k, \bar{x}_k : 1 \leq k \leq n\}$, the reduction f must create $\langle S_\phi, t_\phi\rangle$ such that

$$\langle\phi\rangle \in \text{3-CNF-SAT} \iff \langle S_\phi, t_\phi\rangle \in \text{SS}.$$

f creates a set S_ϕ of $2n + 2m$ numbers, each with $n + m$ decimal digits:

$$S_\phi = \{\underbrace{\boxed{n+m} \quad \boxed{n+m} \quad \boxed{n+m} \quad \cdots \quad \boxed{n+m}}_{2n+2m}\}.$$

The n most significant digits correspond to the n variables $x_1, \ldots, x_n$. The m least significant digits correspond to the m clauses.

CRUCIAL PROPERTY: the sum of all digits in each position i, $1 \leq i \leq n + m$, does never generate a carry.

This is important to relate the satisfiability of ϕ to the achievement of the target t_ϕ.

With no loss of generality:

1. Assume that each variable x_k of $\phi(x_1, \ldots, x_n)$ appears in at least one clause (as $y_j^i = x_k$ or $y_j^i = \bar{x}_k$).
2. Assume that no clause contains x_k and $\bar{x}_k$.

Otherwise: Eliminate variable (1) or clause (2) ($x_k \vee \bar{x}_k = 1$!).

We create:

- two numbers per variable x_i: δ_i, δ_i', $1 \leq i \leq n$ ($2n$).
- two numbers per clause C_j: s_j, s_j', $1 \leq j \leq m$ ($2m$).

$\forall 1 \leq i \leq n$:

- δ_i, δ_i' have a single 1 in the ith position of the n most significant digits.
- δ_i has 1's in all positions j of the m least significant digits, where C_j contains literal x_i.
- δ_i' has 1's in all positions j of the m least significant digits, where C_j contains literal $\bar{x}_i$.
- All other digits are 0.

$\forall 1 \leq j \leq m$:

- s_j has a single 1 in the jth position of the m least significant ones.
- s_j' has a single 2 in the jth position of the m least significant ones.
- All other digits are 0.

Finally, $t_\phi = \underbrace{1 \cdots 1}_{n} \underbrace{44 \cdots 4}_{m}$.

$n = 2$, $m = 2$: 1144.

(Recall that all numbers have to be interpreted to the base 10.)

EXAMPLE:

$$\phi(x_1, x_2, x_3) = \underbrace{(x_1 \vee \bar{x}_2 \vee x_3)}_{C_1} \wedge \underbrace{(x_1 \vee x_2 \vee x_3)}_{C_2}.$$

S_ϕ:

	x_1	x_2	x_3	C_1	C_2	
δ_1	1	0	0	1	1	10.011
δ_1'	1	0	0	0	0	10.000
δ_2	0	1	0	0	1	1.001
δ_2'	0	1	0	1	0	1.010
δ_3	0	0	1	1	1	111
δ_3'	0	0	1	0	0	100
s_1	0	0	0	1	0	10
s_1'	0	0	0	2	0	20
s_2	0	0	0	0	1	1
s_2'	0	0	0	0	2	2
t_ϕ	1	1	1	4	4	11.144

There are exactly 3 1's in each clause column (the δ_i, δ_i' rows).

The sum in each clause column is 3 (the s_j, s_j' rows).

Observe that the sum of all numbers yields

$$\underbrace{22 \cdots 2}_{n} \underbrace{66 \cdots 6}_{m}.$$

Two 1's due to δ_i and δ_i' in the ith column.

3 1's due to literals plus 1 and 2 due to s_j & s_j' in the jth column.

No carry can ever be generated by summing any subset of numbers $\Rightarrow$ no "interference" between different columns.

The running time of the reduction is proportional to the number of digits generated:

$$(\underbrace{2n}_{\delta_i, \delta_i'} + \underbrace{2m}_{s_j, s_j'} + \underbrace{1}_{t_\phi}) \cdot (n + m) = O((n + m)^2) = O(|\langle \phi \rangle|^2).$$

NOTE: After we generate the numbers in decimal, we can encode them in binary without changing the order of magnitude of the size of the encoding.

Let us prove that

$$\langle \phi(x_1, \ldots, x_n) = \bigwedge_{j=1}^{m} C_j \rangle \in \text{3-CNF-SAT} \iff f(\langle \phi \rangle) = \langle S_\phi, t_\phi \rangle \in \text{SS}.$$

($\Rightarrow$) $\phi \in$ 3-CNF-SAT $\Rightarrow \exists \vec{b} : \phi(\vec{b}) = 1$. Let us build a subset $S'_\phi \subseteq S_\phi : \sum_{s \in S'_\phi} s = t_\phi$.

For each $1 \leq i \leq n$:

- if $b_i = 1$ then $\delta_i \in S'_\phi$;
- if $b_i = 0$ then $\delta'_i \in S'_\phi$.

After this selection of n values:

1. The sum of the values in S'_ϕ will have 1's in the n most significant digits (exactly one between δ_i and δ'_i is selected for $1 \leq i \leq n$).
2. The digit of the sum in each clause column j (m least significant digits) reflects the number of true literals in C_j under $\vec{b}$ (δ_i has 1's for clauses containing x_i, while δ'_i has 1's for clauses containing $\bar{x}_i$).

Observe that since $\phi(\vec{b}) = 1$, thanks to 2), each sum digit in clause positions is > 0! (either 1, 2, or 3).

Then $\forall j$, $1 \leq j \leq m$, I can "adjust" the digit in the jth position by adding to S'_ϕ:

1. s_j, if the sum digit is 3 (3 true literals);
2. s'_j, if the sum digit is 2 (2);
3. s_j AND s'_j, if the sum digit is 1 (1).

Now, in $\sum_{s \in S'_\phi} s$, the digit in each of the m least significant positions is 4!

Thus,

$$\sum_{s \in S'_\phi} s = \underbrace{1 \cdots 1}_{n} \underbrace{4 \cdots 4}_{m} = t_\phi,$$

therefore $f(\langle \phi \rangle) = \langle S_\phi, t_\phi \rangle \in$ SS.

EXAMPLE:

$$\phi(x_1, x_2, x_3) = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

Let $\vec{b} = (1, 0, 1) \Rightarrow \phi(\vec{b}) = 1$.

We put $\delta_1, \delta'_2, \delta_3$ in S'_ϕ:

$$\{\delta_1 = 10011, \delta'_2 = 01010, \delta_3 = 00111\}.$$

The sum in the column relative to

C_1 is $3 \Rightarrow$ we add $s_1 = 00010$.

C_2 is $2 \Rightarrow$ we add $s'_2 = 00002$.

$$\sum_{s \in S'_\phi} s = \begin{array}{rcl} 10011 & + & \delta_1 + \\ 1010 & + & \delta'_2 + \\ 111 & + & \delta_3 + \\ 10 & + & s_1 + \\ 2 & & s'_2 + \\ \hline 11144 & & = t_\phi \end{array}$$

($\Leftarrow$) Let $f(\langle\phi\rangle) = \langle S_\phi, t_\phi\rangle \in$ SS then $\exists S'_\phi : \sum_{s\in S'_\phi} s = t_\phi$.

1. Since the sum digit in each of the n most significant digits is 1, there is either δ_i or δ'_i in S'_ϕ, for $1 \le i \le n$. Let

$$b_i = \begin{cases} 1 & \text{if } \delta_i \in S'_\phi, \\ 0 & \text{if } \delta'_i \in S'_\phi. \end{cases}$$

In each clause column j, since the sum digit is 4, there is a contribution ≥ 1 coming from δ_i or δ'_i, for all $1 \le i \le n$ (or otherwise the sum digit would be $\le 1 + 2 = 3$!).

$\Rightarrow$ under $\vec{b}$ there is at least one true literal for each clause

$$\Rightarrow \phi(\vec{b}) = 1 \Rightarrow \langle\phi\rangle \in \text{3-CNF-SAT}.$$

EXAMPLE:

$$\phi(x_1, x_2, x_3) = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

Let $S'_\phi = \{\delta_1, \delta'_2, \delta_3, s_1, s'_2\}$. Then

10011	+	δ_1+
01010	+	δ'_2+
00111	+	δ_3+
00010	+	s_1+
00002	=	s'_2 =
11144		$= t_\phi$

We built $\vec{b} = (1, 0, 1)$. Under $\vec{b}$ there are 3 true literals in C_1 (only $s_1 \in S'_\phi$) and 2 true literals in C_2 (only $s'_2 \in S'_\phi$))

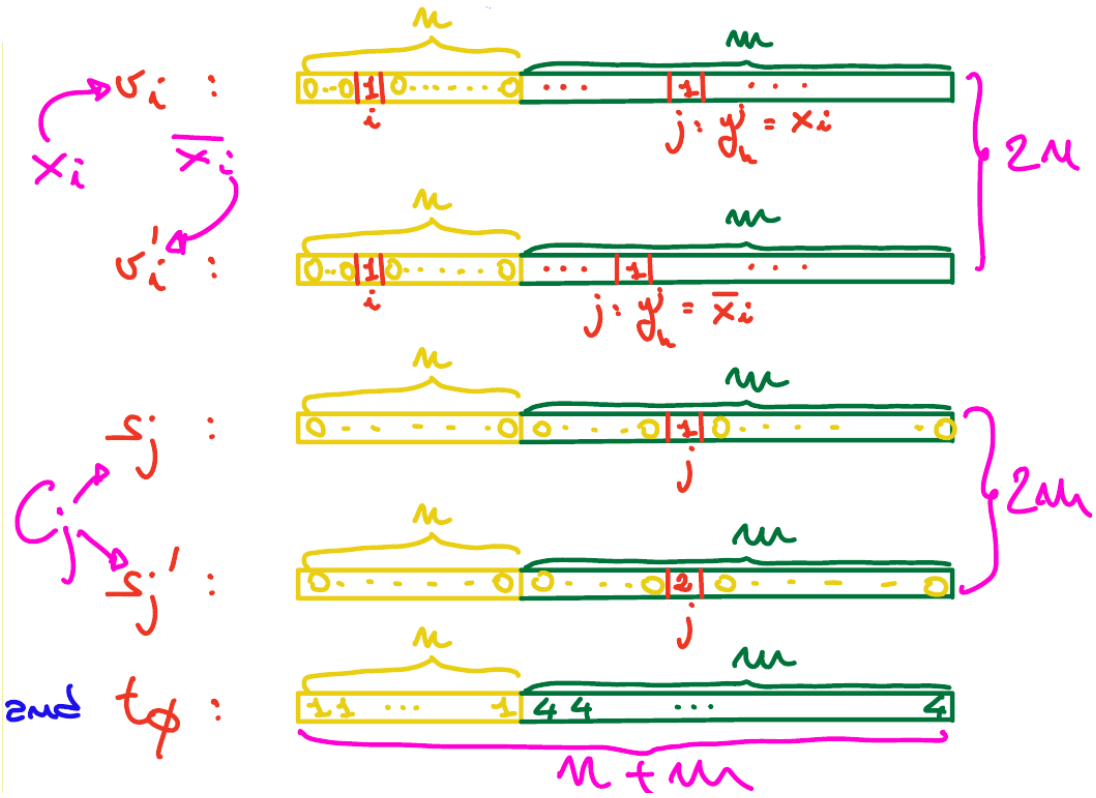
3-CNF-SAT $\le_p$ SUBSET SUM (SS)

$$(\langle S, t\rangle;\ \exists S' \subseteq S : \sum_{s\in S'} s = t\ ?)$$

$$\langle\phi(x_1, \ldots, x_n) = C_1 \wedge \cdots \wedge C_m\rangle \xrightarrow{f} \langle S_\phi, t_\phi\rangle$$

$$S_\phi = \{\delta_i, \delta'_i : 1 \le i \le n;\ s_j, s'_j : 1 \le j \le m\}.$$

$2n + 2m$ numbers (REPRESENTED IN BASE 10).



Crucial property: In each position, the sum of the digits of all numbers in S_ϕ is ≤ 6. NO CARRIES!

IDEA: t_ϕ achievable if and only if each clause satisfiable under same $\vec{b}$.

EXAMPLE:

$$\phi(x_1, x_2, x_3) = \underbrace{(x_1 \vee \bar{x}_2 \vee x_3)}_{C_1} \wedge \underbrace{(x_1 \vee x_2 \vee x_3)}_{C_2}.$$

S_ϕ:

	n			m		
	x_1	x_2	x_3	C_1	C_2	
δ_1	1	0	0	1	1	10.011
δ'_1	1	0	0	0	0	10.000
δ_2	0	1	0	0	1	1.001
δ'_2	0	1	0	1	0	1.010
δ_3	0	0	1	1	1	111
δ'_3	0	0	1	0	0	100
s_1	0	0	0	1	0	10
s'_1	0	0	0	2	0	20
s_2	0	0	0	0	1	1
s'_2	0	0	0	0	2	2
t_ϕ	1	1	1	4	4	11.144

$2n$: there are exactly 3 1's in each clause position and exactly 2 1's in each variable position.

$2m$: the sum in each clause position is 3 (0 in variable positions).

NOTE:

$$\sum_{i=1}^{n}(\delta_i + \delta'_i) + \sum_{j=1}^{m}(s_j + s'_j) = (\underbrace{2\cdots 2}_{n}\underbrace{6\cdots 6}_{m})_{10}.$$

NO CARRIES!

Let us prove that

$$\langle \phi(x_1, \ldots, x_n) = \bigwedge_{j=1}^{m} C_j \rangle \in \text{3-CNF-SAT} \iff f(\langle \phi \rangle) = \langle S_\phi, t_\phi \rangle \in \text{SS}.$$

($\Rightarrow$) $\phi \in$ 3-CNF-SAT $\Rightarrow \exists \vec{b} : \phi(\vec{b}) = 1$. Let us build a subset $S'_\phi \subseteq S_\phi : \sum_{s \in S'_\phi} s = t_\phi$.

For each $1 \leq i \leq n$:

- if $b_i = 1$ then $\delta_i \in S'_\phi$;
- if $b_i = 0$ then $\delta'_i \in S'_\phi$.

After this selection of n values:

1. The sum of the values in S'_ϕ will have 1's in the n most significant digits (exactly one between δ_i and δ'_i is selected for $1 \leq i \leq n$).
2. The digit of the sum in each clause column j (m least significant digits) reflects the number of true literals in C_j under $\vec{b}$ (δ_i has 1's for clauses containing x_i, while δ'_i has 1's for clauses containing $\bar{x}_i$).

Observe that since $\phi(\vec{b}) = 1$, thanks to 2), each sum digit in clause positions is > 0! (either 1, 2, or 3).

Then $\forall j$, $1 \leq j \leq m$, I can "adjust" the digit in the jth position by adding to S'_ϕ:

1. s_j, if the sum digit is 3 (3 true literals);
2. s'_j, if the sum digit is 2 (2);
3. s_j AND s'_j, if the sum digit is 1 (1).

Now, in $\sum_{s \in S'_\phi} s$, the digit in each of the m least significant positions is 4!

Thus,

$$\sum_{s \in S'_\phi} s = \underbrace{1 \cdots 1}_{n} \underbrace{4 \cdots 4}_{m} = t_\phi,$$

therefore $f(\langle \phi \rangle) = \langle S_\phi, t_\phi \rangle \in$ SS.

EXAMPLE:

$$\phi(x_1, x_2, x_3) = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

Let $\vec{b} = (1, 0, 1) \Rightarrow \phi(\vec{b}) = 1$.

We put $\delta_1, \delta'_2, \delta_3$ in S'_ϕ:

$$\{\delta_1 = 10011, \delta'_2 = 01010, \delta_3 = 00111\}.$$

The sum in the column relative to

C_1 is $3 \Rightarrow$ we add $s_1 = 00010$.

C_2 is $2 \Rightarrow$ we add $s'_2 = 00002$.

$$\sum_{s \in S'_\phi} s = \begin{array}{rcl} 10011 & + & \delta_1 + \\ 1010 & + & \delta'_2 + \\ 111 & + & \delta_3 + \\ 10 & + & s_1 + \\ 2 & & s'_2 + \\ \hline 11144 & & = t_\phi \end{array}$$

($\Leftarrow$) Let $f(\langle \phi \rangle) = \langle S_\phi, t_\phi \rangle \in$ SS then $\exists S'_\phi : \sum_{s \in S'_\phi} s = t_\phi$.

1. Since the sum digit in each of the n most significant digits is 1, there is either δ_i or δ'_i in S'_ϕ, for $1 \leq i \leq n$. Let

$$b_i = \begin{cases} 1 & \text{if } \delta_i \in S'_\phi, \\ 0 & \text{if } \delta'_i \in S'_\phi. \end{cases}$$

In each clause column j, since the sum digit is 4, there is a contribution ≥ 1 coming from δ_i or δ'_i, for all $1 \leq i \leq n$ (or otherwise the sum digit would be $\leq 1 + 2 = 3$!).

$\Rightarrow$ under $\vec{b}$ there is at least one true literal for each clause

$$\Rightarrow \phi(\vec{b}) = 1 \Rightarrow \langle \phi \rangle \in \text{3-CNF-SAT}.$$

EXAMPLE:

$$\phi(x_1, x_2, x_3) = (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

Let $S'_\phi = \{\delta_1, \delta'_2, \delta_3, s_1, s'_2\}$. Then

$$\begin{array}{lll} 10011 & + & \delta_1 + \\ 01010 & + & \delta'_2 + \\ 00111 & + & \delta_3 + \\ 00010 & + & s_1 + \\ \underline{00002} & = & s'_2 = \\ 11144 & & = t_\phi \end{array}$$

We built $\vec{b} = (1, 0, 1)$. Under $\vec{b}$ there are 3 true literals in C_1 (only $s_1 \in S'_\phi$) and 2 true literals in C_2 (only $s'_2 \in S'_\phi$).

EXERCISE

Design a decision algorithm A_L for a generic $L \in$ NP and analyze its running time.

EXERCISE

Consider HALTING-PROBLEM (RAM version) (HP-RAM).

I: $\langle C, x \rangle$, $C(x)$ RAM pseudocode with parameter $x \in \{0,1\}^*$.

Q: Does $C(x)$ terminate?

Prove that HP-RAM $\in$ NPH.

EXERCISE

Prove the following proposition:

$$\underbrace{(L' \leq_p L)}_{\text{HP1}} \wedge \underbrace{(L \in \text{NP})}_{\text{HP2}} \Rightarrow L' \in \text{NP}.$$

EXERCISE

Consider this decision problem:

TESTING

I: $\langle C_1(x_1, \ldots, x_n), C_2(x_1, \ldots, x_n) \rangle$.

Q: $\exists \vec{b} \in \{0,1\}^n : C_1(\vec{b}) \neq C_2(\vec{b})$?

Prove that TESTING $\in$ NPH.

EXERCISE

Consider the following problem:

KNAPSACK

I: $\langle a_1, a_2, \ldots, a_n, b;\ c_1, c_2, \ldots, c_n, K\rangle$,

a_i**:** weights of n items;
b**:** total weight bound (capacity);
c_i**:** profit/value of the n items;
K**:** lower bound on total profit ($K \in \mathbb{N}$).

Q: Is there a subset of items of total weight $\leq b$ and total profit $\geq K$?

Prove that KNAPSACK $\in$ NPH.

EXERCISE (Complementary languages)

Let $L^c = \{0,1\}^* - L$. Prove that $\forall L \subseteq \{0,1\}^*$:

$$\underbrace{(L \in \text{NPC})}_{\text{HP1}} \wedge \underbrace{(L^c \in \text{NP})}_{\text{HP2}} \Rightarrow \underbrace{L^c \in \text{NPC}}_{\text{TH}}.$$

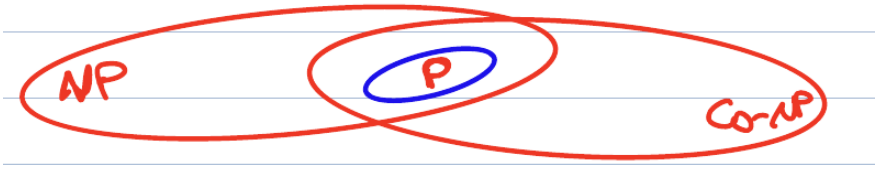
EXERCISE

Define

$$\text{co-NP} = \{L \subseteq \{0,1\}^* : L^c \in \text{NP}\}.$$

$\equiv\ L' = L^c \longrightarrow (\equiv \{L'^c \subseteq \{0,1\}^* : L' \in \text{NP}\})$.

1. Prove that P $\subseteq$ NP $\cap$ co-NP.



2. Prove that if NPC $\cap$ co-NP $\neq \varnothing \Rightarrow$ NP = co-NP.

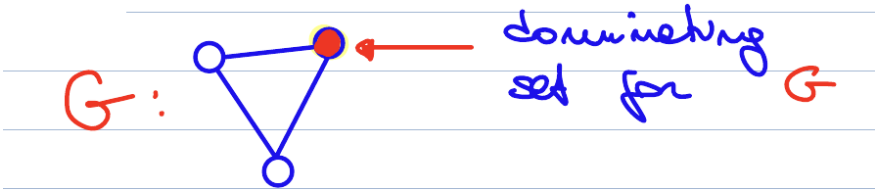
EXERCISE

DEF: Given an undirected graph $G = (V, E)$, a **dominating set** $V' \subseteq V$ is such that

$$\forall v \in V' : (v \in V') \vee (\exists \{u, v\} \in E : u \in V').$$

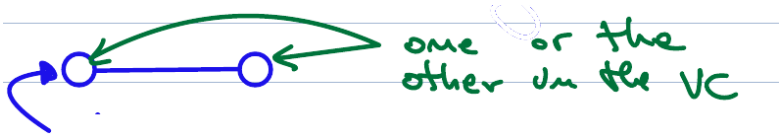
In other words, a node is either in the dominating set or adjacent to a node in the dominating set.

EXAMPLE:



Observe that a dominating set is not a vertex-cover.

VICEVERSA: In a graph with no isolated nodes, a vertex cover is also a dominating set:



(each vertex is the endpoint of an edge: either it is in the vertex cover or the other endpoint must be for the edge to be covered).

DOMINATING SET (DS)

I: $\langle G = (V, E), K \rangle$.

Q: Does G have a DS of size K?

Prove that DS $\in$ NPH.

EXERCISE

Consider the following problem:

L-PATH

I: $\langle G = (V, E), u, v, K \rangle$, $G = (V, E)$ undirected graph, $u, v \in V$, $1 \leq K \leq |V|$.

Q: Is there a simple path from u to v of length $\geq K$?

(decision problem for LONGEST-PATH).

Prove that HAMILTON $\leq_p$ L-PATH (thus L-PATH $\in$ NPH).

EXERCISE

Assume that there exists a polynomial algorithm $A_{\text{SS}}(\langle S, t \rangle)$ for SS:

I: $\langle S, t \rangle : S \subseteq \mathbb{N}$ finite, $t \in \mathbb{N}$.

Q: $\exists S' \subseteq S : \sum_{s \in S'} s = t$?

Design a polynomial algorithm $A_{\text{SUBSET}}(\langle S, t \rangle)$ which uses A_{SS} as a subroutine and, in case $\langle S, t \rangle \in$ SS, returns $S' : \sum_{s \in S'} s = t$.

EXERCISE

Consider the following problem:

PARTITION

I: $\langle S \rangle$, $S \subset \mathbb{N}$, finite.

Q: $\exists S_1, S_2 \subset S$, $(S_1 \cup S_2 = S) \wedge (S_1 \cap S_2 = \varnothing)$:

$$\sum_{s \in S_1} s = \sum_{s \in S_2} s \ ?$$

Show that PARTITION $\in$ NPH.

APPROXIMATION ALGORITHMS

- NPC decision problems often correspond to hard optimization problems of huge practical impact: we cannot give up on them!

Different approaches:

1. Non-polynomial algorithms: sometimes suitable for "small" instances. Interesting subcategory: pseudopolynomial algorithms: complexity polynomial in the values (and NOT the size of their encoding).

EXAMPLE: Using dynamic programming we can obtain an algorithm for SS with running time $O(|S| \cdot t)$ (homework! $|S| \times t$ boolean table A:

$$A(i,j) = 1 \iff \exists S' \subseteq \{s_1, \ldots, s_i\} : \sum_{s \in S'} s = j,$$

$1 \leq i \leq |S|$, $0 \leq j \leq t$. Solution in $A[|S|, t]$. Work out recurrence).

$|S| \cdot t$ is polynomial in the value t but NOT in the length of its encoding $O(\log t)$.

2. Restriction of the instance set: solve the problem on a subset of instances for which there is a polynomial algorithm.

EXAMPLE:

$$\text{2-CNF-SAT} \in \text{P}$$

$$\text{TREE-VERTEX-COVER} \in \text{P}$$

$$\text{SMALL-TARGET SUBSET SUM} \in \text{P} \qquad \left(t \in O\left(\log^k \left(\sum_{s \in S} s \right) \right) \right).$$

3. "Intelligent" exhaustive search (O.R. approach): e.g. branch-and-bound: try to get optimal solution while avoiding to examine all feasible solutions (pruning strategies).

Heuristic methods: no upper bound on worst-case running time.

Other heuristics: local-search, tabu-search, simulated annealing: may return nonoptimal solution (local optimum) ($\rightarrow$ see OR2).

4. APPROXIMATION ALGORITHMS: return a feasible solution whose cost is within a given error margin from the optimal solution in worst-case polynomial time.

EXAMPLE: Return, in linear time, a vertex cover of size at most twice the size of the minimum vertex cover.

3 vs 4: We want hard guarantees on time and solution quality.

(Sometimes a solution obtained through 4. is used as a starting solution for the heuristics of 3.)

Let Π be an optimization problem with positive integer cost function:

$$\Pi \subseteq \mathcal{I} \times \mathcal{S}, \qquad \forall i \in \mathcal{I} : \mathcal{S}(i) = \{s \in \mathcal{S} : i \Pi s\},$$

$$C : \mathcal{S} \to \mathbb{N} - \{0\}, \qquad \text{and let}$$

$$s^*(i) = \arg\max\{C(s) : s \in \mathcal{S}(i)\}.$$

DEF: Let A_Π be an algorithm such that $\forall i \in \mathcal{I} : A_\Pi(i) \in \mathcal{S}(i)$. We say that A_Π is a $\rho(n)$-approximation algorithm for Π if $\forall i \in \mathcal{I}$, $|i| = n$:

$$\max\left\{\frac{C(s^*(i))}{C(A_\Pi(i))}, \frac{C(A_\Pi(i))}{C(s^*(i))}\right\} \leq \underbrace{\rho(n)}_{\text{approximation ratio}}.$$

OBSERVATION 1: The above max is always ≥ 1: $\Rightarrow \rho(n) \geq 1$, $\forall n$. Better approximation achieved for $\rho(n)$ closer to 1. ($\forall n : \rho(n) = 1$: EXACT ALGORITHM).

OBSERVATION 2: For maximization problems the above max is always the first term, and for minimization problems the second. Equivalently:

$$C(A_\Pi(i)) \geq \frac{C(s^*(i))}{\rho(n)} \qquad \text{MAX. PROBLEMS (factor } \rho(n)),$$
$$C(A_\Pi(i)) \leq C(s^*(i)) \cdot \rho(n) \quad \text{MIN. PROBLEMS (factor } \rho(n)).$$

An approximation algorithm finds feasible solutions of controlled quality. Also, it provides bounds on the optimal solution.

In particular:

$$C(s^*(i)) \leq C(A_\Pi(i))\rho(n) \qquad \text{MAX.}$$

upper bound on $C(s^*(i))$.

$$C(s^*(i)) \geq \frac{C(A_\Pi(i))}{\rho(n)} \qquad \text{MIN.}$$

lower bound on $C(s^*(i))$.

DEF: We say that Π is $\rho(n)$-approximable in polynomial time if there exists a $\rho(n)$-approximation algorithm A_Π for Π running in polynomial time.

Hard optimization problems are not all equal w.r.t. approximation: some are $\rho = O(1)$-approximable (in poly. time), some only feature $\rho(n)$ with $\lim_{n\to\infty} \rho(n) = \infty$.

We can often prove that if P $\neq$ NP there cannot exist a poly-time $\rho(n)$-approximation algorithm for some function $\rho(n)$. (INAPPROXIMABILITY RESULTS).

EXAMPLE: If P $\neq$ NP there cannot exist a

$$\rho(|V|) = \text{ⓗ}(|V|^{1-\varepsilon})$$

poly-time approximation algorithm for MAX-CLIQUE, for any fixed ε [Arora et al. 1990].

There are also "easier hard" problems which admit very "precise" approximation algorithms:

DEF: An approximation scheme for an optimization problem Π is a two-input algorithm

$$A_\Pi(i, \varepsilon) : \quad i \in \mathcal{I}, \quad \varepsilon > 0$$

such that, for any fixed ε, $A_\Pi(i, \varepsilon)$ is a $\rho(n) = (1 + \varepsilon)$-approximation algorithm for Π.

DEF: An Approximation Scheme is Polynomial-Time (PTAS) if, for any fixed ε, $A_\Pi(i, \varepsilon)$ runs in time polynomial in $n = |i|$.

Observe that under the above definition, running times such as

$$O(n^{1/\varepsilon})$$

are polynomial in n (for fixed ε).

This is not completely satisfactory: e.g. if I want to reduce the "error" ε by a factor k:

$$O(n^{1/\varepsilon}) \rightsquigarrow O(n^{1/(\varepsilon/k)}) = O(n^{k/\varepsilon})!$$

Time exponential in the improvement.

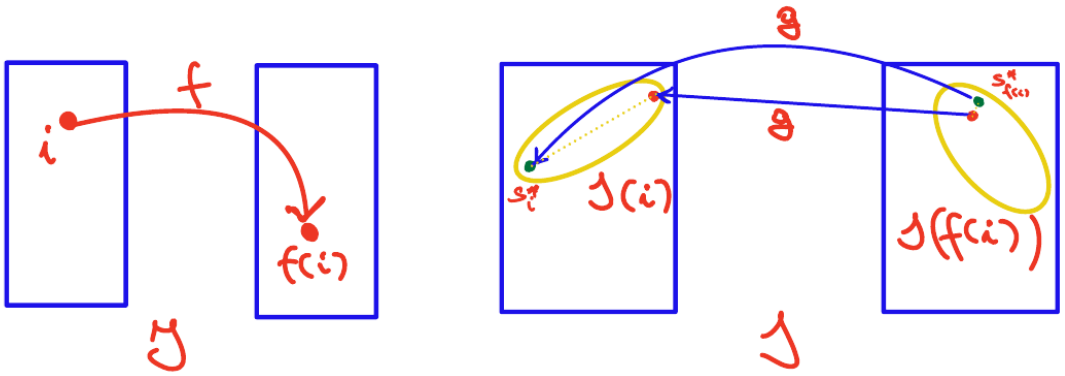
It would be desirable that a constant-factor improvement in quality yields a constant-factor increase in running time:

DEF: An Approximation Scheme is Fully-Polynomial-Time (FPTAS) if the running time of $A_\Pi(i, \varepsilon)$ is polynomial both in $n = |i|$ and $\frac{1}{\varepsilon}$.

E.g. $T_{A_\Pi}(n, \varepsilon) = O\left(\frac{1}{\varepsilon^2} n^3\right)$.

Morale: While optimization problems associated to NPC problems are all equivalent w.r.t. exact solution, they behave differently w.r.t. approximation.

Intuition: The idea of reducibility can be extended to optimization problems using an extra function g, mapping each element of $\mathcal{S}(f(i))$ into an element of $\mathcal{S}(i)$:



Function g maps optimal solutions in $\mathcal{S}(f(i))$ into optimal solutions in $\mathcal{S}(i)$, however the mapping does not necessarily preserve the approximation quality!

A TWO-APPROXIMATION ALGORITHM FOR (MINIMUM) VERTEX COVER

Optimization version: given $G = (V, E)$ undirected, determine its vertex cover $V^* \subseteq V$

$$(\forall \{u, v\} \in E : (u \in V^*) \vee (v \in V^*))$$

of minimum size.

We will prove that a simple greedy approach yields a good quality solution.

IDEA:

1. **GREEDY CHOICE:** consider an arbitrary edge $\{u, v\}$ and ADD BOTH u and v to the vertex cover. (OBS: NOT INTUITIVE!)
2. **CLEAN-UP:** remove all covered edges from E: $E' = E - \{e \in E : \{u, v\} \cap e \neq \varnothing\}$.

APPROX-VERTEX-COVER $(G = (V, E))$

$V' \leftarrow \varnothing$
$E' \leftarrow E$
while $(E' \neq \varnothing)$ **do**
    * select $\{u, v\} \in E'$ arbitrary * GC
    $V' \leftarrow V' \cup \{u\} \cup \{v\}$
    $E' \leftarrow E' - \{e \in E' : \{u, v\} \cap e \neq \varnothing\}$ * C.U.
return V'

The algorithm correctly returns a vertex cover (I eliminate an edge only when it is covered by some selected node).

Running time: $O(|V| + |E|) = O(|\langle G = (V, E)\rangle|)$ (every time I select edge $\{u, v\}$, I eliminate the two adjacency lists of u and v).

TOUGH PART OF THE ANALYSIS: approximation ratio.

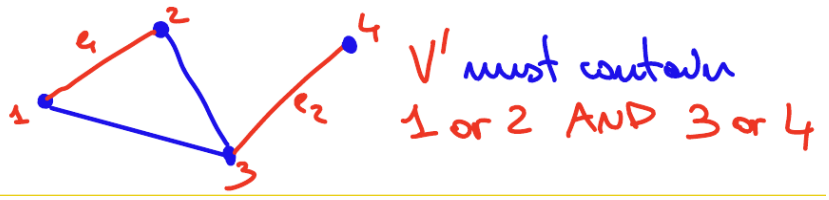
Let A be the set of edges selected by A-V-C (G) at each iteration of the while loop.

PROPERTY: $\forall e_1 \neq e_2 \in A : e_1 \cap e_2 = \varnothing$. The edges in A do not share endpoints (since at each iteration all edges sharing one endpoint with the selected edge are removed).

A set of edges A not sharing endpoints is called a **matching**.

FACT: If a graph $G = (V, E)$ contains a matching $A \subseteq E$, then for any vertex-cover $V' \subseteq V$ we have $|V'| \geq |A|$.

PROOF: There must be at least one endpoint of each edge e in A in V', or otherwise e would not be covered.



COROLLARY: $|V^*| \geq |A|$.

Algorithm A-V-C (G) returns a vertex cover V', with $|V'| = 2|A|$.

We have

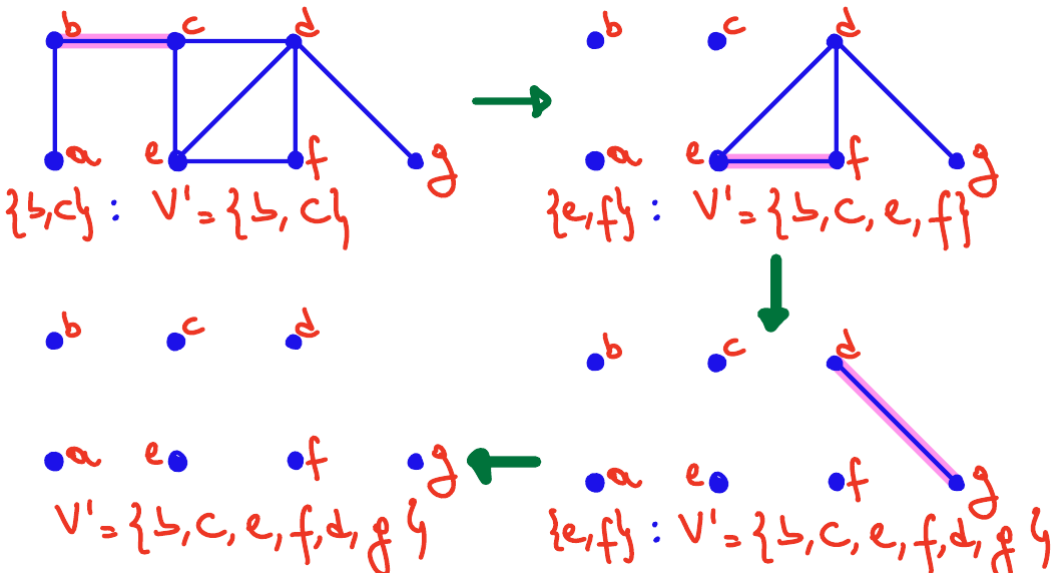
$$\rho = \underbrace{\frac{|V'|}{|V^*|}}_{\geq |A|} \leq \frac{2|A|}{|A|} = 2.$$

IMPORTANT: To prove an upper bound to ρ, we always need:

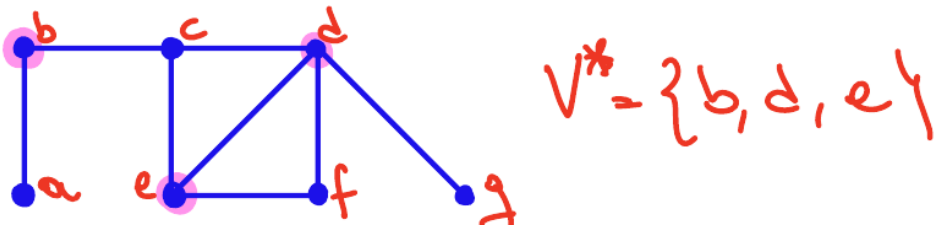
- a lower bound to $C(s^*)$ — MINIMIZATION;
- an upper bound to $C(s^*)$ — MAXIMIZATION.

We can show that the bound on the approximation ratio is tight by exhibiting an instance for which

$$C(A_\Pi(i)) = \rho \cdot C(s^*(i)) \qquad \text{(MIN)}.$$



$|V'| = 6$ but:



$\Rightarrow |V'| = 2|V^*|$ (tight approximation!).

LESSONS LEARNED: To get a bound on the approximation we need to establish explicit bounds (lower or upper) on $C(s^*)$.

For VC, we establish such a (lower) bound by relating $|V^*|$ (cannot be computed) to the size of an easily obtainable construct (a matching A of G).

We will see that this approach can be used for other problems.

OBSERVATION: The matching A constructed implicitly by A-V-C (G) is maximal, in the sense that $\forall e \in E : A \cup \{e\}$ is not a matching — (PROVE IT!).

EXERCISE: Prove that the set of all nodes forming the edges of a maximal matching are a vertex cover.